

PONTIFICIA UNIVERSIDAD JAVERIANA
Facultad de Ingeniería – Departamento de Ingeniería de Sistemas

DOCUMENTO DE DISEÑO DE SOFTWARE (SDD)

Generación de Escenarios de Movilidad Urbana para la Toma de Decisiones en Localidades de Bogotá

Estándar base: IEEE Std 1016

Samuel Esteban Campos
Erick Salazar Suárez
Santiago Chitiva Contreras
Felipe Andrés Garrido Flores

Director: Ing. Andrés Oswaldo Calderón Romero, PhD.

Bogotá D.C., 11 de noviembre de 2026
Versión 2.1

Historial de cambios

Tabla 1: Historial de cambios del documento.

Versión	Fecha	Descripción del cambio	Autor(es)
0.1	20/02/2026	Creación inicial del documento de diseño de software. Definición de su estructura y organización de las secciones principales.	Samuel Campos
0.2	27/02/2026	Desarrollo de la introducción, incluyendo el propósito del documento, alcance y convenciones utilizadas para describir el diseño del sistema.	Erick Salazar
0.3	06/03/2026	Descripción general del sistema, identificación de sus principales funcionalidades y definición de las características que debe cumplir.	Santiago Chitiva
0.4	13/03/2026	Primera definición de la arquitectura del sistema y distribución de responsabilidades entre los módulos principales.	Felipe Garrido
0.5	20/03/2026	Ampliación de la descripción arquitectónica mediante la identificación de componentes y sus relaciones dentro de la solución.	Samuel Campos, Erick Salazar
0.6	10/04/2026	Incorporación del diseño de datos, identificación de las entidades principales y definición de las relaciones necesarias para representar la información del sistema.	Santiago Chitiva
0.7	24/04/2026	Desarrollo del diseño de componentes, especificando las responsabilidades y operaciones principales de cada elemento del sistema.	Felipe Garrido, Samuel Campos
0.8	08/05/2026	Incorporación del diseño de interfaces y descripción de la comunicación entre los componentes de la solución.	Erick Salazar
0.9	22/05/2026	Definición inicial de los atributos de calidad del sistema, considerando aspectos como rendimiento, mantenibilidad, disponibilidad y escalabilidad.	Santiago Chitiva, Felipe Garrido
1.0	12/06/2026	Consolidación de la primera versión del diseño de software. Revisión de la arquitectura, estructura de datos, componentes e interfaces.	Samuel Campos, Erick Salazar
1.1	17/07/2026	Actualización de la arquitectura para reflejar los ajustes realizados en el procesamiento de datos y la integración con el simulador SUMO.	Felipe Garrido
1.2	14/08/2026	Actualización del diseño de componentes e interfaces para incorporar las funcionalidades de parametrización y visualización de escenarios desde el módulo web.	Santiago Chitiva, Samuel Campos
1.3	11/09/2026	Ampliación del diseño de datos y componentes relacionados con el análisis de intersecciones y la configuración de programas semafóricos.	Erick Salazar, Felipe Garrido
1.4	25/09/2026	Incorporación de los elementos arquitectónicos del simulador 3D comparativo y de los componentes asociados a la evaluación de seguridad vial.	Santiago Chitiva

Versión	Fecha	Descripción del cambio	Autor(es)
1.5	24/10/2026	Actualización general del documento para reflejar el alcance ampliado del proyecto, incluyendo ajustes en arquitectura, datos, componentes, interfaces y atributos de calidad.	Samuel Campos, Felipe Garrido
2.1	11/11/2026	Reestructuración completa del documento para ajustarlo a la plantilla de Descripción del Diseño del Software: introducción, arquitectura (vistas lógica, física y de procesos), diseño detallado (estructura, comportamiento, persistencia e interfaz de usuario) y referencias. El contenido previo sobre arquitectura, datos, componentes, interfaces, decisiones y limitaciones se reubicó en las nuevas secciones.	Erick Salazar

Tabla de Contenidos

1 Introducción 4
1.1 Propósito y contenido del documento 4
1.2 Audiencia 4
1.3 Alcance del sistema 4
1.4 Glosario y acrónimos 5
2 Arquitectura 6
2.1 Vista lógica del sistema 6
2.2 Vista física del sistema 13
2.3 Vista de procesos del sistema 14
3 Diseño detallado 17
3.1 Estructura del sistema 17
3.2 Comportamiento del sistema 29
3.3 Persistencia 36
3.4 Interfaz de usuario 40
4 Referencias 43

Lista de Figuras

Figura 1 Diagrama de componentes: librerías, entornos de ejecución y software externo. 7
Figura 2 Diagrama de despliegue: nodos, entornos de ejecución y artefactos instalados. 13
Figura 3 Diagrama de actividad P1: simular un escenario de demanda. 15
Figura 4 Diagrama de actividad P2: consultar y ajustar un semáforo existente. 16
Figura 5 Diagrama de actividad P3: evaluar en 3D un cruce sin semáforo. 17
Figura 6 Diagrama de clases del visor: contenedor, hooks, cliente de la API y paneles. 18
Figura 7 Diagrama de clases del simulador 3D. 20
Figura 8 Diagrama de clases de la API y de la orquestación de corridas. 21
Figura 9 Diagrama de clases de la comparación 3D. 23
Figura 10 Diagrama de clases del ETL: configuraciones, demanda parametrizada y resultado del pipeline. 24
Figura 11 Diagrama de clases del análisis de intersecciones. 27
Figura 12 Diagrama de clases del semáforo propuesto, la escena 3D y la telemetría. 28
Figura 13 Diagrama de secuencia de una simulación parametrizada. 30
Figura 14 Diagrama de estados de una corrida. 31

Figura 15 Diagrama de secuencia de la preparación de una comparación 3D. 32
Figura 16 Diagrama de secuencia de la ejecución en vivo de la comparación 3D. 33
Figura 17 Diagrama de clases con estereotipos del modelo de persistencia. 37
Figura 18 Diagrama de navegación de la interfaz gráfica. 41
Figura 19 Esquema de la pantalla principal del visor. 42
Figura 20 Esquema del simulador 3D comparativo. 43

Lista de Tablas

Tabla 1 Historial de cambios del documento. 1
Tabla 2 Módulos del sistema cubiertos por este documento. 4
Tabla 3 Glosario de términos y acrónimos. 5
Tabla 4 Componentes del sistema, sus responsabilidades e interfaces. 7
Tabla 5 Interfaces entre los componentes del sistema. 8
Tabla 6 Endpoints de la API. 9
Tabla 7 Mensajes del WebSocket `/comparacion3d/{id}`. 11
Tabla 8 Librerías, máquinas virtuales y software de terceros requeridos. 11
Tabla 9 Capas del patrón arquitectónico por capas. 12
Tabla 10 Nodos físicos y componentes instalados en cada uno. 13
Tabla 11 Pasos del proceso P1. 15
Tabla 12 Pasos del proceso P2. 16
Tabla 13 Pasos del proceso P3. 17
Tabla 14 Clases del visor y su explicación. 18
Tabla 15 Asociaciones del diagrama del visor. 19
Tabla 16 Clases del simulador 3D y su explicación. 20
Tabla 17 Clases de la API y de la orquestación de corridas. 21
Tabla 18 Asociaciones del diagrama de la API. 22
Tabla 19 Clases de la comparación 3D. 23
Tabla 20 Clases y módulos del ETL. 24
Tabla 21 Asociaciones del diagrama del ETL. 26
Tabla 22 Clases y módulos del análisis de intersecciones. 27
Tabla 23 Clases y módulos del semáforo propuesto, la escena 3D y la telemetría. 28
Tabla 24 Mensajes relevantes de la simulación parametrizada. 30
Tabla 25 Mensajes relevantes de la preparación. 32
Tabla 26 Elementos del modelo de persistencia. 37
Tabla 27 Columnas del TSV de viajes y su uso. 38
Tabla 28 Campos de `estado.json` (dataclass `Corrida`). 38
Tabla 29 Esquema de cada *feature* de `kepler.geojson`. 39
Tabla 30 Ubicaciones de almacenamiento y su política. 40
Tabla 31 Transiciones del diagrama de navegación. 41
Tabla 32 Elementos de la pantalla principal. 42
Tabla 33 Elementos del simulador 3D. 43

1 Introducción

1.1 Propósito y contenido del documento

Este Documento de Descripción del Diseño del Software (SDD) describe cómo está construido el sistema *Generación de Escenarios de Movilidad Urbana – Usaquén*. Sigue la estructura de la norma IEEE Std 1016-2009 [1] y presenta la arquitectura según la norma ISO/IEC/IEEE 42010 [2] y el modelo de vistas «4+1» de Kruchten [3]. Complementa a la Especificación de Requisitos (SRS, [4]), que dice *qué* debe hacer el sistema, y al Plan de Gestión (SPMP, [5]), que dice *cómo* se gestiona el proyecto.

El documento se organiza en dos partes:

- **Arquitectura** (Sección 2): la estructura y el comportamiento del sistema con un alto nivel de abstracción, en tres vistas: lógica (componentes, librerías e interfaces), física (hardware y dónde se instala cada componente) y de procesos (lo que hace el usuario con el sistema).
- **Diseño detallado** (Sección 3): el diseño de bajo nivel, es decir, las clases de cada componente, la secuencia de las acciones complejas, la forma en que se guardan los datos y las pantallas de la interfaz gráfica.

El documento describe el sistema tal como está implementado en la rama de desarrollo (SRS versión 2.1). Donde algo está diseñado pero no construido, o construido con una limitación conocida (por ejemplo, el transporte público), se indica de forma explícita.

1.2 Audiencia

Este documento está dirigido a:

- **El equipo de desarrollo**, como referencia para mantener y extender el código.
- **El director del proyecto** y **los jurados evaluadores**, como evidencia de las decisiones de diseño y de su justificación.
- **Futuros mantenedores o investigadores** que quieran reutilizar el pipeline de simulación con otra localidad de Bogotá.

1.3 Alcance del sistema

El sistema se compone de dos módulos que se documentan en conjunto:

Tabla 2: Módulos del sistema cubiertos por este documento.

Módulo	Descripción	Directorio
`simulation-service`	Paquete Python instalable (≥ 3.11). Contiene el pipeline ETL que convierte la Encuesta de Movilidad en entradas de SUMO, ejecuta la simulación y exporta los resultados, y una API REST/WebSocket (FastAPI) que lo expone.	`simulation-service/`
`kepler`	Aplicación web de una sola página (Vite + React) que embebe kepler.gl para animar los viajes simulados y ofrece paneles de demanda, intersecciones, semaforización y el simulador 3D comparativo.	`kepler/`

El resto del repositorio (`notebooks/`, `data/`) es un espacio de exploración de datos (CRISP-DM, fases 1 y 2) cuyos productos entran al sistema como insumos versionados; no forma parte del diseño de software y solo se menciona como fuente de datos.

Dentro del alcance. El sistema:

- Transforma los viajes de la Encuesta de Movilidad de Bogotá 2023 [6] (filtrados a los internos de Usaquén, ventana 07:00–10:00) en demanda multimodal para SUMO [7], replicando cada viaje según su factor de expansión.
- Permite parametrizar la demanda (presets de cambio modal, conteos por modo, semilla, estilo de conducción y cierres viales) y ejecutar la simulación como trabajos en segundo plano con avance y cancelación.
- Sirve resultados precalculados de los presets para consulta inmediata y visualiza los viajes animados sobre un mapa.
- Cataloga las intersecciones semaforizadas de la red, muestra sus accesos, volúmenes y programas, y calcula un programa de Webster [8].
- Prioriza los cruces sin semáforo que justificarían uno nuevo y los compara en 3D, en tiempo real, con y sin un semáforo propuesto, con telemetría de riesgo [9].

Fuera del alcance.

- **Transporte público real**: no hay buses, paradas ni GTFS; esos viajes se simulan como caminata (`<person><walk>`).
- **Calibración y validación estadística** del modelo frente a aforos reales.
- **Programas semafóricos reales**: los programas son los que genera `netconvert` desde OpenStreetMap, no los de la Secretaría Distrital de Movilidad.
- **Autenticación, multiusuario y despliegue en la nube**: es una herramienta local de investigación.
- Otras localidades distintas de Usaquén (el diseño es parametrizable por escenario, pero solo existe el escenario `default`).

1.4 Glosario y acrónimos

Tabla 3: Glosario de términos y acrónimos.

Término	Significado
SUMO	*Simulation of Urban MObility*: simulador microscópico de tráfico de código abierto. Se usa a través de `sumo`, `duarouter`, `netconvert`, `cutRoutes.py` y `randomTrips.py`.
TraCI	*Traffic Control Interface*: protocolo para controlar una instancia de SUMO en ejecución (avanzar pasos, consultar y modificar actores).
FCD	*Floating Car Data*: salida de SUMO con la posición de cada actor en cada periodo. Es la fuente de las exportaciones.
TAZ	*Traffic Analysis Zone*: zona de análisis de tráfico; origen y destino de los viajes (derivadas de las UPZ de Usaquén).
UPZ / ZAT	Unidad de Planeamiento Zonal / Zona de Análisis de Transporte de Bogotá.
OD	Matriz origen–destino.
`fexp`	Factor de expansión de la encuesta: cuántas personas representa un viaje observado.
Preset	Regla de cambio modal predefinida (`encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`) que redistribuye los viajes entre modos.
Corrida (*run*)	Ejecución del pipeline con una demanda concreta; sus archivos viven en `runs/<id>/`.
Precalculada	Corrida de un preset con conteos por defecto, guardada en el repositorio para servirla sin simular.

Término	Significado
`vType`	Tipo de vehículo de SUMO; aquí hay uno por modo de viaje (`TransportMode`).
`tlLogic`	Programa de un semáforo en SUMO (fases, duraciones, estados por movimiento).
Cruce	Agrupación de nodos de SUMO próximos que forman una intersección real (p. ej. las dos calzadas de una avenida).
Webster	Método de F. V. Webster para el ciclo óptimo de un semáforo.
SSM	*Surrogate Safety Measures*: indicadores de riesgo que no esperan a un choque (TTC, DRAC, PET).
TTC / DRAC / PET	Tiempo a la colisión / tasa de desaceleración para evitarla / tiempo posterior a la invasión del cruce.
Ego	Actor (vehículo o peatón) seguido por la cámara en el simulador 3D; se inyecta con el mismo id y ruta en los dos lados.
Lado	Una de las dos instancias de SUMO del simulador 3D: A = sin semáforo, B = con semáforo propuesto.
Kepler / kepler.gl	Biblioteca de visualización geoespacial; su capa *Trip* anima rutas con marca de tiempo.
ETL	*Extract, Transform, Load*: aquí, encuesta → trips → rutas → simulación → GeoJSON.
SDM	Secretaría Distrital de Movilidad de Bogotá (fuente de la encuesta y de los siniestros).
OSM	OpenStreetMap: fuente de la red vial, nombres de calles, POI y huellas de edificios.

2 Arquitectura

Este capítulo describe la arquitectura del sistema, entendida como la definición de su estructura y comportamiento con un alto nivel de abstracción [1], [2]. Se presenta con tres vistas del modelo «4+1» de Kruchten [3] (y la estructura del documento de arquitectura de RUP [10]): la **vista lógica** (los componentes de software y sus interfaces), la **vista física** (el hardware y dónde se instala cada componente) y la **vista de procesos** (las acciones que el usuario realiza sobre el sistema). El detalle de bajo nivel de cada componente está en el Sección 3.

2.1 Vista lógica del sistema

La vista lógica identifica los componentes más gruesos del sistema, sin considerar en qué máquina se instalan, junto con las interfaces que los conectan. Incluye también las librerías, los entornos de ejecución y el software externo que el sistema necesita para funcionar.

2.1.1 Diagrama de componentes

El sistema es una aplicación cliente–servidor local. El navegador carga el visor (`kepler`); el servidor de desarrollo de Vite reenvía `/api/*` (HTTP y WebSocket) a la API de `simulation-service`, que orquesta las herramientas de SUMO como subprocesos o mediante TraCI y lee y escribe en el sistema de archivos.

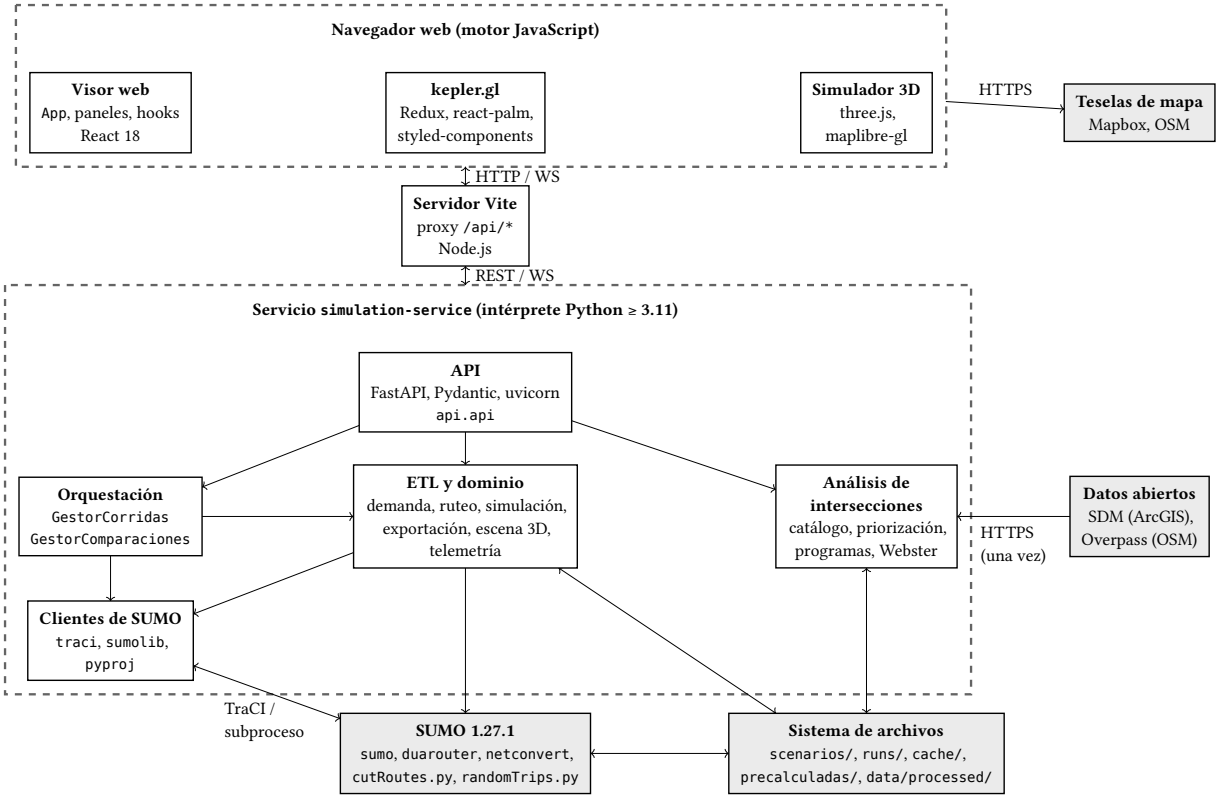


Figura 1: Diagrama de componentes: librerías, entornos de ejecución y software externo.

2.1.2 Descripción de los componentes

Tabla 4: Componentes del sistema, sus responsabilidades e interfaces.

Componente	Responsabilidad	Interfaces
Visor web	Dueño del mapa y de los paneles de usuario: descarga y anima los viajes, edita la demanda, consulta intersecciones y abre el simulador 3D.	**Requiere:** API REST y WebSocket (vía `/api`), teselas de mapa. **Ofrece:** la interfaz gráfica.
kepler.gl	Biblioteca de visualización geoespacial; su capa *Trip* anima los viajes con marca de tiempo. Impone Redux para su estado.	**Requiere:** GeoJSON de `LineString` con `[lon, lat, z, t]`.
Simulador 3D	Dos escenas three.js (sin y con semáforo propuesto), HUD y minimapa; se carga de forma diferida.	**Requiere:** `GET .../geometria` y `WS /comparacion3d/{id}`.
Servidor Vite	Sirve el visor y reenvía `/api/*` a `localhost:8000`, retirando el prefijo.	**Ofrece:** HTTP y WebSocket del mismo origen para el navegador.
API	Valida las peticiones (Pydantic), traduce errores a códigos HTTP y publica los recursos REST y el WebSocket. No contiene lógica de negocio pesada.	**Ofrece:** REST/JSON y WebSocket (Sección 2.1.3.1). **Requiere:** Orquestación, ETL y Análisis.

Componente	Responsabilidad	Interfaces
Orquestación	`GestorCorridas` ejecuta una corrida pesada a la vez, con avance y cancelación; `GestorComparaciones` prepara y corre las comparaciones 3D (máximo 2).	**Requiere:** `run_pipeline`, TraCI y el sistema de archivos.
ETL y dominio	Pipeline encuesta → viajes → rutas → simulación → GeoJSON; demanda parametrizada; semáforo propuesto, demanda de zona, escena 3D y telemetría de riesgo.	**Requiere:** SUMO y el sistema de archivos. **Ofrece:** funciones `run_pipeline`, `run_escenario_semaforo`.
Análisis de intersecciones	Catálogo de semáforos de la red, priorización de cruces sin semáforo, lectura y validación de programas `tlLogic` y cálculo de Webster.	**Requiere:** red SUMO, evidencia versionada y volúmenes de una corrida.
Clientes de SUMO	Librerías Python que hablan con SUMO: `traci` (control en vivo), `sumolib` (puertos libres y lectura de redes) y `pyproj` (UTM a lon/lat).	**Requiere:** binarios de SUMO.
SUMO	Simulador microscópico (`sumo`), ruteador (`duarouter`), constructor de redes (`netconvert`) y herramientas Python (`cutRoutes.py`, `randomTrips.py`).	**Ofrece:** línea de comandos y TraCI por socket local.
Sistema de archivos	Almacén único de datos: insumos, corridas, cachés y precalculadas (Sección 3.3).	Lectura y escritura local.
Teselas de mapa	Mapbox (base de kepler.gl, requiere `VITE_MAPBOX_TOKEN`) y OSM (minimapa del 3D, sin token).	HTTPS desde el navegador.
Datos abiertos	SDM (siniestros) y Overpass de OSM (POI, edificios, nombres de vías). Solo se consultan al ejecutar `python -m etl.sumo.evidencia`; los resultados se versionan, así que el sistema funciona sin red.	HTTPS desde el servicio.

2.1.3 Interfaces entre componentes

Tabla 5: Interfaces entre los componentes del sistema.

Interfaz	Medio	Uso
Visor ↔ API	HTTP/JSON y WebSocket, por el proxy de Vite (`/api`)	Toda la comunicación del visor; el detalle está en Sección 2.1.3.1.
API ↔ Orquestación y ETL	Llamadas a funciones de Python, con inyección de dependencias (`Depends`)	Los gestores se sustituyen en las pruebas con `app.dependency_overrides`.
Orquestación ↔ ETL	`run_pipeline(..., on_progreso, cancelar)`	Único punto de entrada de una corrida. `on_progreso(etapa, avance | None)` y `cancelar` (`threading.Event`) cruzan todas las capas hasta el subproceso.
ETL ↔ SUMO	Subprocesos (`sumo`, `duarouter`, `netconvert`) y TraCI por socket local	Binarios del paquete `eclipse-sumo` en el `PATH` del entorno virtual. `cutRoutes.py` y `randomTrips.py` se buscan en `$SUMO_HOME/tools` y, si no, en el paquete pip.
Servicio ↔ Sistema de archivos	Lectura y escritura local	Insumos, corridas y cachés (Sección 3.3).
Hilo de simulación ↔ hilo de eventos	`asyncio.Queue` con `call_soon_threadsafe`; `queue.Queue` para comandos	`correr_comparacion` corre en un hilo y publica frames; los comandos del cliente entran por la cola (pausa, velocidad y aceleración se leen directamente del objeto `Comparacion`).
Navegador ↔ teselas	HTTPS	Mapbox (token en `kepler/.env`, ignorado por Git) y OSM.
Servicio ↔ datos abiertos	HTTPS (solo al ejecutar `python -m etl.sumo.evidencia`)	SDM (ArcGIS FeatureServer) y Overpass con un espejo de respaldo.

2.1.3.1 API del servicio de simulación

La API es REST sobre JSON más un WebSocket. Se sirve en `http://localhost:8000` (documentación interactiva autogenerada en `/docs`). El visor la usa a través del prefijo `/api`, que el proxy de Vite retira. Los errores usan `{"detail": ...}` con los códigos: `404` (no existe), `409` (recurso no listo o en estado incompatible), `422` (validación), `429` (límite de comparaciones abiertas).

Tabla 6: Endpoints de la API.

Método y ruta	Códigos	Descripción
`GET /health`	200	Comprobación de vida: `{"msg": "healthy"}`.
`GET /demanda/modos`	200	Modos de viaje con etiqueta, color y conteo por defecto.
`GET /demanda/presets`	200	Presets con descripción y conteos por defecto.
`GET /demanda/estilos`	200	Estilos de conducción con sus parámetros de SUMO.
`POST /simulaciones`	202, 422	Cuerpo: `{preset, conteos{modo:n}, semilla?, estilo, cierres[]}`.

Método y ruta	Códigos	Descripción
		Encola una corrida; devuelve su estado inicial con `id`.
`GET /simulaciones`	200	Historial, la más reciente primero.
`GET /simulaciones/{id}`	200, 404	Estado, avance y resultado de una corrida.
`POST /simulaciones/{id}/cancelar`	202, 404, 409	Cancela una corrida en cola o en curso.
`GET /simulaciones/{id}/kepler-trips`	200, 404, 409	GeoJSON de trips de una corrida terminada.
`GET /precalculadas`	200	Presets ya simulados (metadatos).
`GET /precalculadas/{preset}/kepler-trips`	200, 404	Trips comprimidos (`Content-Encoding: gzip`, `X-Tamano-Original`).
`GET /simulation-output?limit=`, `GET /kepler-trips`	200, 404	Salidas de la corrida por defecto (herencia del pipeline por línea de comandos).
`GET /intersecciones?run_id=&precalculada=`	200, 404, 409	Semáforos como GeoJSON de puntos, con volumen de la corrida.
`GET /intersecciones/{tl_id}`	200, 404	Ficha: accesos, fases, volúmenes, notas sobre el origen del programa.
`GET /intersecciones/priorizacion`	200	Cruces sin semáforo por nivel y puntaje, umbrales, notas y descartes.
`GET /intersecciones/{tl_id}/programas[/{pid}]`	200, 404	Lista o detalle de programas `tlLogic`.
`PUT /intersecciones/{tl_id}/programas/{pid}`	200, 404, 422	Valida y guarda un programa editado en el escenario.
`POST /intersecciones/{tl_id}/programas/webster`	200, 422	Calcula ciclo y verdes de Webster con los volúmenes de una corrida.
`POST /intersecciones/{tl_id}/escenarios`	202, 404	Recorta red y demanda de una zona (radio 50–2.000 m).
`GET /escenarios/{run_id}`	200, 404	Estado de un recorte de zona.
`POST /intersecciones/{tl_id}/comparaciones`, `GET /comparaciones/{id}`	202, 200, 404	Compara varios programas de un semáforo por métricas de corridas de zona.
`POST /cruces/{nodo}/comparacion3d`	202, 422, 429	Cuerpo: `{lon, lat, run_id?, intensidades?, nodos?}`; intensidades 0–3.000 por hora. Prepara la comparación en segundo plano.
`GET /comparacion3d`, `GET /comparacion3d/{id}`	200, 404	Lista y estado (`preparando`, `lista`, `error`) con la etapa en curso.
`GET /comparacion3d/{id}/geometria`	200, 404, 409	Geometría de la escena; solo cuando la comparación está `lista`.

Método y ruta	Códigos	Descripción
`DELETE /comparacion3d/{id}`	204, 404	Cierra la comparación y borra sus archivos.
`WS /comparacion3d/{id}`	101, cierre 1008	Transmisión en vivo (ver abajo). Se rechaza si no existe, no está lista o ya hay una ventana conectada.

Ejemplo: creación de una simulación.

```
POST /simulaciones
{"preset": "mas_peatones",
 "conteos": {"auto": 3000, "peaton": 12000, "bici": 2500, "moto": 1557},
 "semilla": 42, "estilo": "normal", "cierres": []}

202 Accepted
{"id": "20261015-101533-a1b2c3", "estado": "en_cola", "avance": 0.0, ...}
```

2.1.3.2 Protocolo del WebSocket de la comparación 3D

Los mensajes son JSON de texto. Una sola ventana puede estar conectada a la vez; el servidor envía un `ping` cada 5 s de inactividad.

Tabla 7: Mensajes del WebSocket `/comparacion3d/{id}`.

Sentido	Mensaje	Significado
Servidor → cliente	`{tipo:"estado", mensaje}`	Progreso del arranque y del calentamiento.
	`{tipo:"frame", t, dt, ego, manual, lados:{a, b}}`	Un paso de simulación con actores, luces, métricas y telemetría (ver Sección 3.3.2.6).
	`{tipo:"ego", id}`	Se inyectó un nuevo actor seguido.
	`{tipo:"giro", direccion}`	Se aceptó un giro en modo manual.
	`{tipo:"aviso", mensaje}`	Comando no aplicable (p. ej. giro imposible desde el acceso actual).
	`{tipo:"fin"}`, `{tipo:"error", mensaje}`, `{tipo:"ping"}`	Fin de la simulación, fallo (con el *stderr* de SUMO) y latido.
Cliente → servidor	`{pausa: bool}`	Pausar o reanudar.
	`{velocidad: x}`	Factor de velocidad, acotado a 0,25–20.
	`{periodo_riesgo: s}`	Cada cuántos segundos simulados se mide el riesgo (0,4–30).
	`{seguir: id}`	Cambiar el actor seguido.
	`{inyectar:{tipo, acceso, giro, estilo}}`	Insertar un actor propio con la ruta del acceso y giro elegidos.
	`{manual: bool}`, `{control: {acelerar:-1|0|1}}`, `{giro:"s"|"l"|"r"}`	Conducción manual: tomar el control, acelerar/frenar y elegir el próximo giro.

2.1.4 Librerías, entornos de ejecución y software requerido

Tabla 8: Librerías, máquinas virtuales y software de terceros requeridos.

Categoría	Software y versión	Uso
Entorno de ejecución	Python ≥ 3.11; Node.js (para Vite); navegador web moderno	Ejecutan el servicio, el servidor de desarrollo y el visor.
Simulación	SUMO 1.27.1 (`eclipse-sumo`), `traci` 1.27.1, `sumolib`	Simulación microscópica, ruteo, construcción de red y control en vivo.
Servicio	FastAPI 0.136.3 (`fastapi[standard]`), Pydantic, `uvicorn`	API REST/WebSocket y validación.
Geodesia	`pyproj` 3.8.0	Coordenadas UTM de la red a lon/lat.
Visor	React 18.3, Vite 6, `vite-plugin-node-polyfills`	Aplicación de una página con proxy `/api`.
Mapa	kepler.gl 3.1 (`@kepler.gl/*`), Redux 4, `react-palm`, `styled-components` 6.1.8, `react-intl`	Mapa y capa Trip.
3D	three.js 0.169, maplibre-gl 4.7.1	Escena comparativa y minimapa OSM.
Calidad y pruebas	`pytest` 9.1.1, `matplotlib` (validación), ESLint 10, pylint	Pruebas y análisis estático.
Datos	Encuesta de Movilidad 2023 (TSV), OSM, siniestros SDM, edificios OSM y Microsoft	Insumos versionados en `data/processed/` y `scenarios/`.

2.1.5 Patrones y decisiones arquitectónicas

Cliente–servidor con servicio monolítico modular. Se eligió un único proceso FastAPI con módulos internos separados (API → orquestación → ETL) en lugar de microservicios, porque el sistema corre en una sola máquina, lo costoso es SUMO y no el enrutamiento, y un solo proceso simplifica la cancelación, el reparto de rutas de archivos y las pruebas.

Tabla 9: Capas del patrón arquitectónico por capas.

Capa	Responsabilidad
Presentación	`kepler/`: mapa kepler.gl, paneles de demanda, ficha de intersección, semaforización y simulador 3D (carga diferida).
Interfaz (API)	`api/api.py`: validación de peticiones, códigos HTTP y WebSocket.
Orquestación	`api/jobs.py` (corridas en segundo plano) y `api/comparacion3d.py` (comparaciones 3D con TraCI).
Dominio y datos	`etl/`: generación de demanda, ruteo, simulación, exportación, análisis de intersecciones y priorización, escena 3D, telemetría.

Dentro del servicio conviven tres estilos adicionales:

- **Tubería de etapas (*pipes and filters*)** para el ETL: cada etapa lee archivos de la anterior y escribe los propios (TSV → `trips.xml` → `rou.xml` → FCD → GeoJSON) y puede ejecutarse sola.
- **Trabajos asíncronos con un solo trabajador** para las corridas largas: la API responde `202 Accepted` y el cliente consulta el estado cada 2 s (*polling*).

- **Flujo en tiempo real por WebSocket** para el simulador 3D: dos instancias de SUMO avanzan en *lockstep* y el servidor emite un *frame* por paso.

Persistencia en archivos. El sistema no usa un motor de base de datos: los resultados de SUMO ya son archivos, hay un único escenario y así se evita instalar y operar otro servicio. **Una instancia.** El estado de las corridas vive en memoria y en disco local, por lo que el sistema no escala en usuarios simultáneos ni tiene autenticación; es una herramienta local de investigación. **SUMO como ejecutable externo.** El servicio solo orquesta procesos y archivos; no integra SUMO como biblioteca.

2.2 Vista física del sistema

La vista física muestra el hardware que interviene y en qué nodo se instala cada componente de la Sección 2.1. El sistema es una herramienta local: casi todo corre en un único equipo (estación de trabajo del usuario o del desarrollador), y solo las teselas de mapa y los datos abiertos vienen de servidores externos.

2.2.1 Diagrama de despliegue

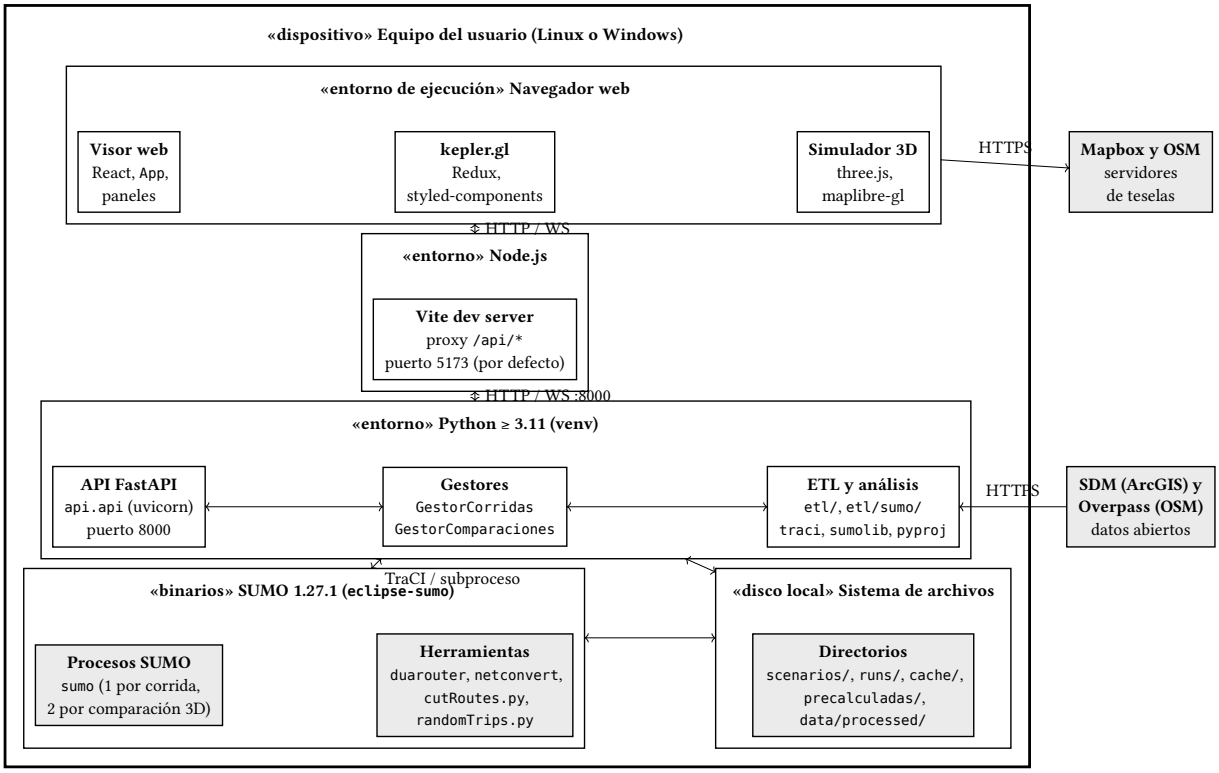


Figura 2: Diagrama de despliegue: nodos, entornos de ejecución y artefactos instalados.

2.2.2 Descripción de los nodos

Tabla 10: Nodos físicos y componentes instalados en cada uno.

Nodo	Componentes instalados	Observaciones
Equipo del usuario (Linux o Windows)	Navegador, Node.js, entorno virtual de Python y binarios de SUMO.	Un solo equipo ejecuta todo. El mismo código corre en Linux y Windows (se contempla el bloqueo de archivos de Windows al borrar el directorio de una comparación y se normalizan los saltos de línea al calcular la huella de insumos).

Nodo	Componentes instalados	Observaciones
Navegador web	Visor web (React), kepler.gl, simulador 3D (three.js, maplibre-gl).	El bundle lo sirve Vite; el simulador 3D se descarga de forma diferida al abrirse. Debe poder manejar la encuesta completa (24.600 viajes, 64 MB de GeoJSON sin comprimir; 11,7 MB comprimido).
Node.js	Servidor de desarrollo de Vite con el proxy `/api/*`.	Retira el prefijo `/api` y reenvía HTTP y WebSocket a `localhost:8000`.
Python ≥ 3.11 (venv)	API (uvicorn), gestores, ETL, análisis y los clientes `traci`, `sumolib` y `pyproj`.	Un solo proceso de API con hilos: un trabajador para las corridas y un hilo por comparación 3D.
Binarios de SUMO	`sumo`, `duarouter`, `netconvert` y las herramientas `cutRoutes.py` y `randomTrips.py`.	Paquete `eclipse-sumo`, que deja los binarios en el `PATH` del entorno virtual. Cada corrida lanza un `sumo`; cada comparación 3D lanza dos (lado A y lado B), que se comunican por sockets locales de TraCI.
Sistema de archivos	`scenarios/`, `runs/`, `cache/`, `precalculadas/`, `data/processed/`.	Ver Sección 3.3. Es la única persistencia del sistema.
Mapbox y OSM (externo)	Servidores de teselas de mapa.	Mapbox exige `VITE_MAPBOX_TOKEN` en `kepler/.env`; OSM (minimapa del 3D) no necesita token.
SDM y Overpass (externo)	ArcGIS FeatureServer y Overpass de OSM.	Solo al ejecutar `python -m etl.sumo.evidencia`; los resultados se versionan, así que el sistema funciona sin red.

Puesta en marcha. Los componentes se instalan y arrancan así:

```
# Servicio (desde simulation-service/)
python3.11 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
python -m etl.etl                 # pipeline completo (varios minutos)
python -m etl.precalculadas       # regenerar presets precalculados
uvicorn api.api:app --reload      # API en :8000, documentación en /docs

# Visor (desde kepler/)
npm install && npm run dev        # requiere la API en :8000 y VITE_MAPBOX_TOKEN
en .env
```

2.3 Vista de procesos del sistema

La vista de procesos muestra las principales acciones con las que el usuario interactúa con el sistema. Se representan con diagramas de actividad (uno por proceso, sin repetir la misma información en otro tipo de diagrama) y son consistentes con el diagrama de navegación de la interfaz gráfica (Figura 18). Las pantallas citadas se describen en la Sección 3.4 y el detalle de las acciones complejas, con diagramas de secuencia, en la Sección 3.2.

Los tres procesos principales son: simular un escenario de demanda (P1), consultar y ajustar un semáforo existente (P2) y evaluar en 3D un cruce sin semáforo (P3).

2.3.1 P1. Simular un escenario de demanda

El usuario elige o ajusta la demanda. Si coincide exactamente con un preset precalculado, la simulación se carga al instante; si no, se lanza una corrida en segundo plano, cuyo avance se consulta hasta que termina.

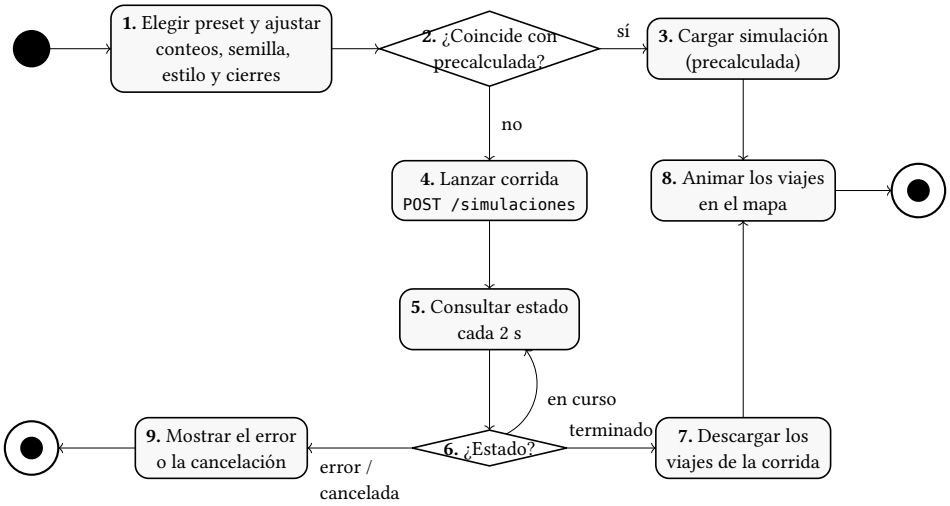


Figura 3: Diagrama de actividad P1: simular un escenario de demanda.

Tabla 11: Pasos del proceso P1.

Paso	Descripción
1	En el panel «Demanda» el usuario selecciona uno de los presets (`encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`), modifica los viajes por modo (el total debe estar entre 1 y 100.000), la semilla y el estilo de conducción, y opcionalmente define cierres viales en el panel «Cierres».
2	El visor compara los valores del panel con las simulaciones precalculadas (`GET /precalculadas`). Si coinciden exactamente con una, el botón principal es «Cargar simulación»; si no, es «Simular».
3	Camino precalculado: el visor descarga `GET /precalculadas/{preset}/kepler-trips`, que llega comprimido; la barra de progreso se calcula con la cabecera `X-Tamano-Original`.
4	Camino con simulación: el visor envía `POST /simulaciones`. La API valida el cuerpo y responde `202` con el `id` de la corrida, que queda en cola.
5	El visor consulta `GET /simulaciones/{id}` cada 2 s y muestra en la tarjeta de actividad la etapa (generando demanda, ruteando, simulando, exportando) y el avance. El usuario puede cancelar con `POST /simulaciones/{id}/cancelar`.
6	Decisión según el estado de la corrida: `terminado`, `error`, `cancelada` o aún en curso (se vuelve al paso 5).
7	Con la corrida terminada, el visor descarga `GET /simulaciones/{id}/kepler-trips`.
8	El visor reemplaza el conjunto de datos de kepler.gl y muestra los viajes animados, coloreados por modo.
9	Si la corrida falló, se muestra el mensaje de error; si se canceló, la corrida queda como cancelada en el historial.

2.3.2 P2. Consultar y ajustar un semáforo existente

Con una simulación cargada, el usuario puede inspeccionar cada semáforo de la red, editar su programa, calcular uno por el método de Webster o comparar programas por sus métricas.

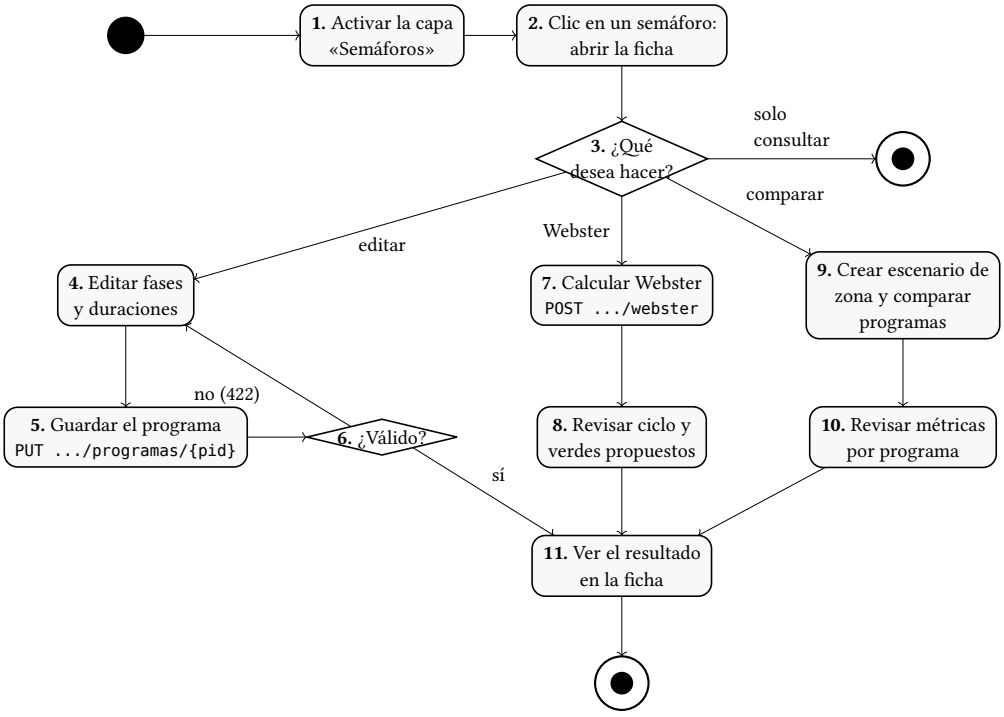


Figura 4: Diagrama de actividad P2: consultar y ajustar un semáforo existente.

Tabla 12: Pasos del proceso P2.

Paso	Descripción
1–2	El conmutador «Semáforos» muestra los semáforos de la red como puntos (`GET /intersecciones`, con el volumen de la corrida cargada). Un clic abre la pestaña «Intersección» con la ficha (`GET /intersecciones/{tl_id}`): accesos, volúmenes, fases y notas sobre el origen del programa. La ficha tiene tres subpestañas: «Ficha» (consultar), «Configurar» (editar y Webster) y «Comparar».
3	Desde la ficha el usuario escoge entre editar el programa, calcular uno de Webster, comparar programas o solo consultar.
4–6	Editar: el usuario cambia fases y duraciones y guarda. La API valida el programa (un estado por movimiento, un amarillo ≥ 3 s si hay verdes, ciclo ≥ 15 s); si no es válido responde `422` y el usuario vuelve a editar.
7–8	Webster: la API calcula el ciclo y los verdes con los volúmenes de una corrida y los devuelve; el usuario los revisa y, si quiere, los edita como un programa nuevo.
9–10	Comparar: el visor pide un escenario de zona (`POST /intersecciones/{tl_id}/escenarios`), se comparan los programas elegidos (`POST .../comparaciones`) y la tabla de comparación muestra las métricas de cada uno.
11	El resultado queda visible en la ficha de la intersección.

2.3.3 P3. Evaluar en 3D un cruce sin semáforo

El usuario identifica los cruces que justificarían un semáforo y compara, en tiempo real, el cruce tal como está contra el mismo cruce con un semáforo propuesto.

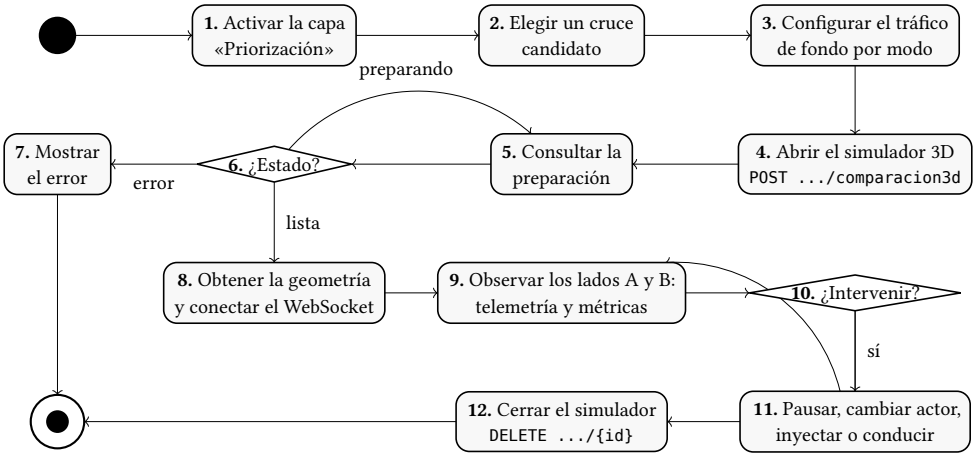


Figura 5: Diagrama de actividad P3: evaluar en 3D un cruce sin semáforo.

Tabla 13: Pasos del proceso P3.

Paso	Descripción
1–2	La capa «Priorización» (`GET /intersecciones/priorizacion`) muestra los cruces sin semáforo por nivel (prioritario, recomendado, observación). El usuario elige uno con un clic en el mapa o desde la lista ordenada por puntaje en la pestaña «Semaforizar».
3	El usuario fija las intensidades del tráfico de fondo por modo (veh/h, 0–3.000), porque los viajes de la encuesta son demasiado escasos en una zona de 350 m.
4	El visor pide la comparación (`POST /cruces/{nodo}/comparacion3d`). La API responde `202` con el `id` y prepara la zona en un hilo; si ya hay dos comparaciones abiertas responde `429`.
5–7	El visor consulta `GET /comparacion3d/{id}` hasta que el estado sea `lista` o `error`; en este último caso se muestra el mensaje.
8	Con la comparación lista, el visor descarga la geometría de la escena y abre el WebSocket. Al conectar, el servidor arranca los dos SUMO y los calienta 240 s simulados.
9	El usuario observa en pantalla dividida el cruce sin semáforo (A) y con el semáforo propuesto (B): el actor seguido, su telemetría de riesgo (TTC, DRAC, PET) y las métricas comparadas por lado.
10–11	El usuario puede pausar, cambiar la velocidad, seguir otro actor, inyectar uno propio (auto, moto, taxi, bici, peatón…) eligiendo acceso y giro, o conducirlo con el teclado. Los comandos se aplican igual en los dos lados.
12	Al cerrar el simulador, el visor elimina la comparación y la API borra sus archivos. Si nadie se conecta en 180 s, la comparación expira.

3 Diseño detallado

Este capítulo muestra los detalles más importantes del diseño de bajo nivel del sistema: la estructura de cada componente de software (clases), el comportamiento de las acciones complejas en el tiempo, la forma en que se almacenan los datos y las pantallas de la interfaz gráfica.

3.1 Estructura del sistema

Esta sección presenta, para cada componente de software de la Sección 2.1, un diagrama de clases con su explicación. Se usa la notación UML con estas convenciones:

- Cada recuadro tiene tres compartimentos (nombre, atributos y métodos). El estereotipo indica el tipo: «dataclass» (clase de datos de Python), «módulo» (funciones agrupadas en un archivo, sin instanciarse), «componente» y «hook» (React), «modelo» (esquema Pydantic) y «enum».
- + es público y - es de uso interno (en el código, con prefijo _).
- Flecha continua: asociación (el origen guarda o usa una referencia al destino). Flecha con rombo: composición (el origen contiene al destino y controla su ciclo de vida). Flecha punteada: dependencia (el origen invoca al destino o lo recibe como parámetro).
- Los atributos se muestran con su tipo de Python o de JavaScript; ? indica que puede ser nulo.

Todos los módulos de `etl/sumo/` siguen un mismo patrón para ser predecibles y fáciles de probar: un `*Config` inmutable (`dataclass(frozen=True)`) con los parámetros y, si invoca una herramienta de SUMO, un método `to_args()`; un `*Result` con lo producido; y un `*Stats` mutable que se pasa y se incrementa durante el proceso. Los envoltorios de subprocesos buscan el binario con `shutil.which` y lanzan una excepción propia (`*Error(RuntimeError)`). Los procesos largos aceptan `on_avance` y `cancelar` y los ejecutan a través de `proceso.ejecutar`, que consulta ambos cada 0,5 s.

3.1.1 Visor web (`kepler/src`)

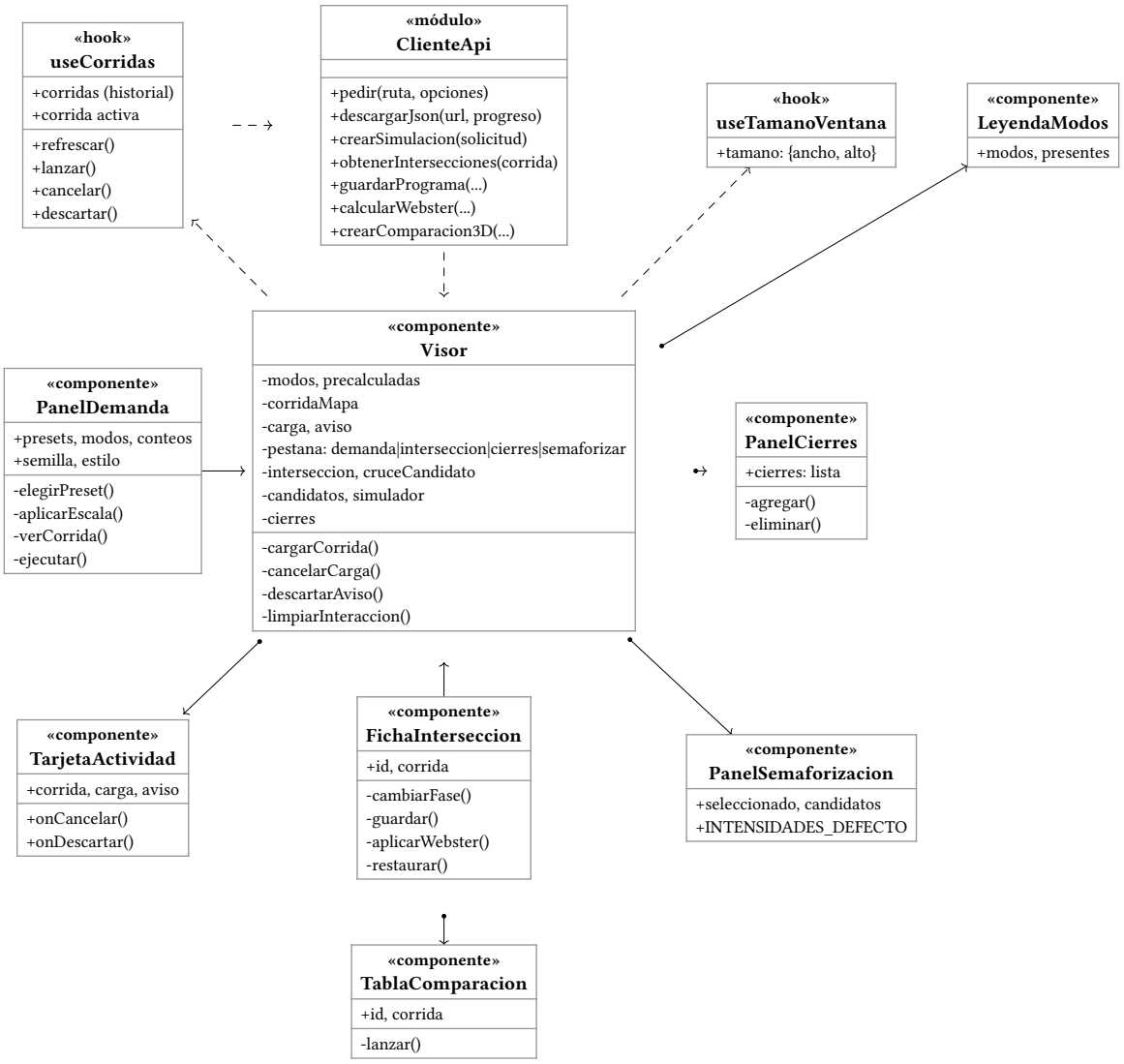


Figura 6: Diagrama de clases del visor: contenedor, hooks, cliente de la API y paneles.

Tabla 14: Clases del visor y su explicación.

Clase	Responsabilidad, atributos y métodos
`Visor` (`App.jsx`)	Dueño del mapa y del estado global de la pantalla. Descarga los viajes con progreso y `AbortController`, reemplaza el conjunto de datos de kepler.gl, activa las capas de semáforos y de priorización y abre la ficha de intersección leyendo `visState.clicked`. **Atributos:** `modos` y `precalculadas` (catálogos de la API), `corridaMapa` (corrida mostrada; nulo es la corrida por defecto), `carga` y `aviso` (progreso y mensaje de la descarga), `pestana` (panel activo), `interseccion` (semáforo abierto), `cruceCandidato` y `candidatos` (priorización), `simulador` (parámetros del 3D abierto) y `cierres`. **Métodos:** `cargarCorrida` baja y muestra los viajes de una corrida; `cancelarCarga` aborta la descarga en curso; `descartarAviso` oculta el aviso; `limpiarInteraccion` cierra la ficha y el candidato elegidos.
`useCorridas`	Hook que maneja el historial de corridas, la corrida activa, la consulta cada 2 s y la cancelación. **Métodos:** `refrescar` consulta la API; `lanzar` crea una corrida; `cancelar` pide cancelarla; `descartar` la quita de la tarjeta de actividad.
`useTamanoVentana`	Hook que mantiene el ancho y alto de la ventana, que kepler.gl necesita de forma explícita (el `body` tiene `overflow: hidden`).
`ClienteApi` (`api.js`)	Cliente HTTP único; todas las rutas son relativas a `/api`. `pedir` ejecuta la petición y convierte los errores de la API en mensajes legibles; `descargarJson` descarga con barra de progreso (usa `X-Tamano-Original` en las precalculadas). Las demás funciones corresponden una a una a los endpoints de la Tabla 6.
`PanelDemanda`	Presets, conteos por modo, escala, semilla y estilo. `elegirPreset` carga los conteos del preset; `aplicarEscala` multiplica los conteos; `verCorrida` muestra una corrida del historial; `ejecutar` decide entre «Simular» y «Cargar simulación».
`PanelCierres`	Define cierres viales (edge y horario). `agregar` y `eliminar` modifican la lista, que viaja en la solicitud de simulación.
`TarjetaActividad`	Tarjeta inferior derecha con barras de progreso de la corrida y de la descarga, avisos y botón de cancelar.
`FichaInterseccion`	Accesos, volúmenes y fases del semáforo, edición del programa (`cambiarFase`, `guardar`, `restaurar`), cálculo de Webster (`aplicarWebster`) y comparación de programas.
`PanelSemaforizacion`	Pestaña «Semaforizar»: lista de cruces candidatos por puntaje, tráfico de fondo por modo (`INTENSIDADES_DEFECTO`) y botón para abrir el simulador 3D.
`LeyendaModos`	Leyenda de colores por modo y conmutador de la capa «Semáforos».
`TablaComparacion`	Compara los programas de un semáforo por sus métricas; `lanzar` pide la comparación a la API.

Tabla 15: Asociaciones del diagrama del visor.

Asociación	Significado
`Visor` ◆⟶ paneles	El visor contiene y renderiza `PanelDemanda`, `PanelCierres`, `PanelSemaforizacion`, `TarjetaActividad`, `FichaInterseccion` y `LeyendaModos`; les pasa propiedades y recibe eventos.
`FichaInterseccion` ◆⟶ `TablaComparacion`	La ficha incluye la tabla de comparación de programas.
`Visor` ⇢ `useCorridas`, `useTamanoVentana`	El visor usa los hooks para el historial de corridas y el tamaño de la ventana.
`Visor`, `useCorridas` ⇢ `ClienteApi`	Todo acceso a la API pasa por el cliente único.

3.1.1.1 Simulador 3D (`kepler/src/components/simulador3d`)

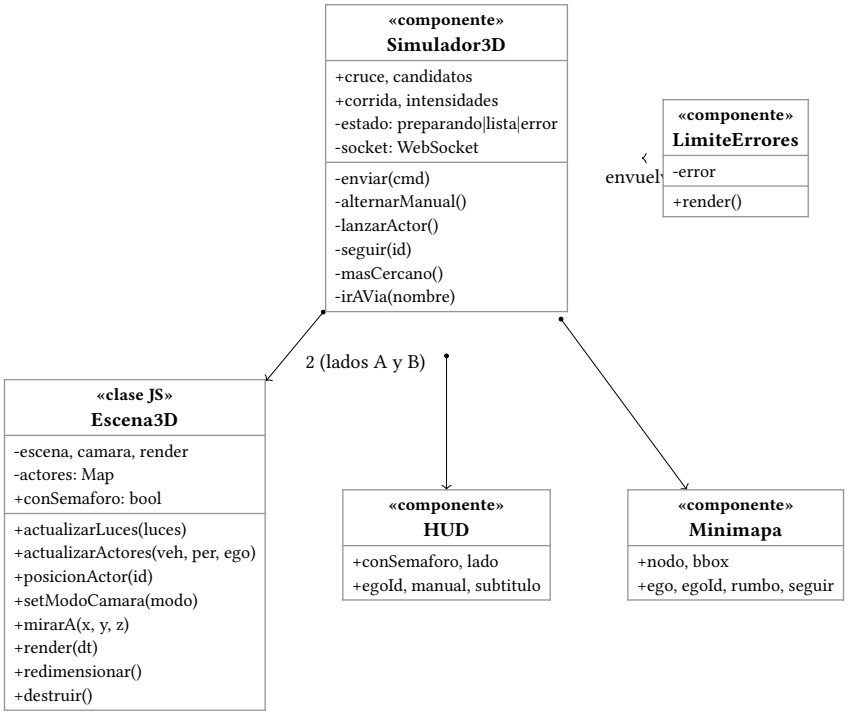


Figura 7: Diagrama de clases del simulador 3D.

Tabla 16: Clases del simulador 3D y su explicación.

Clase	Responsabilidad, atributos y métodos
`Simulador3D`	Ciclo de vida: crear la comparación ⟶ consultar su estado ⟶ obtener la geometría ⟶ conectar el WebSocket; pantalla dividida «sin semáforo» y «con semáforo propuesto». **Atributos:** parámetros recibidos (`cruce`, `candidatos`, `corrida`, `intensidades`), estado de la preparación y socket. **Métodos:** `enviar` manda un comando por el WebSocket; `alternarManual` entra o sale de la conducción manual; `lanzarActor` inyecta un actor propio; `seguir` cambia el actor seguido; `masCercano` elige el actor más próximo al cruce; `irAVia` mueve la vista a una vía por nombre.
`Escena3D` (`escena.js`)	Una escena three.js por lado. Convierte las coordenadas de SUMO a three como $(x, z - z_0, -(y - y_0))$ con el origen en el nodo; funde las mallas estáticas por material; interpola los actores entre frames; ofrece cámara de

Clase	Responsabilidad, atributos y métodos
	persecución con colisión contra edificios o aérea. **Métodos:** `actualizarLuces` fija el estado del semáforo; `actualizarActores` recibe vehículos, personas y el ego de un frame; `posicionActor` devuelve la posición de un actor; `setModoCamara` y `mirarA` controlan la cámara; `render` dibuja un cuadro; `redimensionar` ajusta el lienzo; `destruir` libera recursos de WebGL.
`HUD`	Píldora superior (tiempo, velocidad, pausa) y tarjeta inferior con la telemetría del actor seguido, el índice de riesgo y la comparación de métricas por lado.
`Minimapa`	Minimapa con teselas OSM (maplibre-gl, sin token). maplibre fuerza `position: relative` en su contenedor, por lo que el posicionamiento absoluto vive en un `div` envoltorio.
`LimiteErrores`	Barrera de errores (*error boundary*): un fallo gráfico no tumba el mapa.

Asociaciones. `Simulador3D` contiene dos `Escena3D` (una por lado), un `HUD` y un `Minimapa`, y los alimenta con los frames del WebSocket. `LimiteErrores` envuelve a `Simulador3D`, que se carga de forma diferida (`lazy`) porque three.js y maplibre pesan.

3.1.2 API y orquestación de corridas (`simulation-service/src/api`)

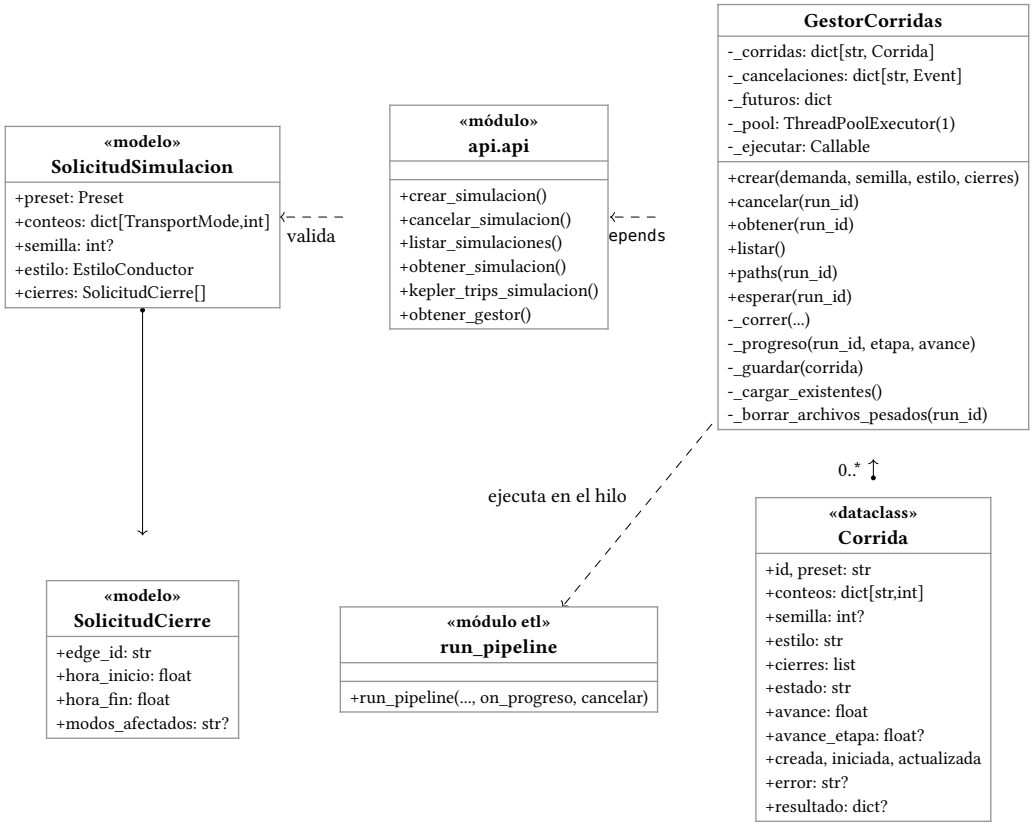


Figura 8: Diagrama de clases de la API y de la orquestación de corridas.

Tabla 17: Clases de la API y de la orquestación de corridas.

Clase	Responsabilidad, atributos y métodos
`SolicitudSimulacion`	Cuerpo de `POST /simulaciones`. Valida con Pydantic que cada conteo esté entre 0 y 100.000 y que el total esté entre 1 y 100.000 viajes (si no,

Clase	Responsabilidad, atributos y métodos
	422), y que la semilla sea ≥ 0. `preset`, `estilo` y las claves de `conteos` son enumerados: rechazan cualquier otro texto.
`SolicitudCierre`	Un cierre vial: `edge_id` del tramo, hora de inicio y fin en segundos, y modos afectados. El ETL lo convierte en un `rerouter` de SUMO.
`api.api`	Aplicación FastAPI: define las rutas HTTP y el WebSocket, traduce errores a códigos HTTP y obtiene los gestores por inyección de dependencias (`obtener_gestor`, `obtener_gestor_comparaciones`), sustituibles en pruebas con `dependency_overrides`. Los métodos mostrados atienden las rutas de corridas de la Tabla 6.
`GestorCorridas`	Cola de un solo trabajador (`ThreadPoolExecutor(1)`): solo una corrida pesada corre a la vez y las demás quedan `en_cola`. **Métodos:** `crear` genera el id (`AAAAMMDD-HHMMSS-<6 hex>`), persiste la corrida y la encola; `cancelar` activa el `threading.Event` de la corrida (si aún no empezaba, la marca `cancelada`; si no, `cancelando`) y lanza `ValueError` si ya terminó; `obtener` y `listar` consultan (la más reciente primero); `paths` devuelve los `ScenarioPaths` de la corrida; `esperar` bloquea hasta que termine (pruebas); `_correr` es el trabajo del hilo: llama a `run_pipeline`, borra el FCD y registra el estado final; `_progreso` traduce el avance de la etapa a la franja global (`FRANJAS_ETAPA`); `_guardar` escribe `estado.json` (como mucho cada 2 s mientras avanza); `_cargar_existentes` reconstruye el historial al arrancar y marca como `error` las corridas que estaban activas («Interrumpida: el servicio se reinició»); `_borrar_archivos_pesados` deja solo `estado.json` en una corrida cancelada.
`Corrida`	Estado persistente de una corrida (se serializa a `estado.json`, Sección 3.3.2.2). El `estado` recorre `en_cola` ⟶ `generando_demanda` ⟶ `ruteando` ⟶ `simulando` ⟶ `exportando` ⟶ `terminado`, o termina en `cancelando`, `cancelada` o `error`.
`run_pipeline`	Función del ETL (Sección 3.1.3), único punto de entrada de una corrida; recibe `on_progreso(etapa, avance|None)` y `cancelar`.

Tabla 18: Asociaciones del diagrama de la API.

Asociación	Significado
`api.api` ⇢ `SolicitudSimulacion`	La ruta valida el cuerpo con el modelo antes de crear la corrida.
`SolicitudSimulacion` ◆⟶ `SolicitudCierre` (0..*)	La solicitud contiene sus cierres viales.
`api.api` ⇢ `GestorCorridas`	La ruta recibe el gestor inyectado con `Depends`.
`GestorCorridas` ◆⟶ `Corrida` (0..*)	El gestor guarda todas las corridas y controla su ciclo de vida.
`GestorCorridas` ⇢ `run_pipeline`	El hilo trabajador invoca el pipeline (inyectable como `_ejecutar` para usar uno falso en las pruebas).

3.1.2.1 Comparación 3D (`api/comparacion3d.py`)

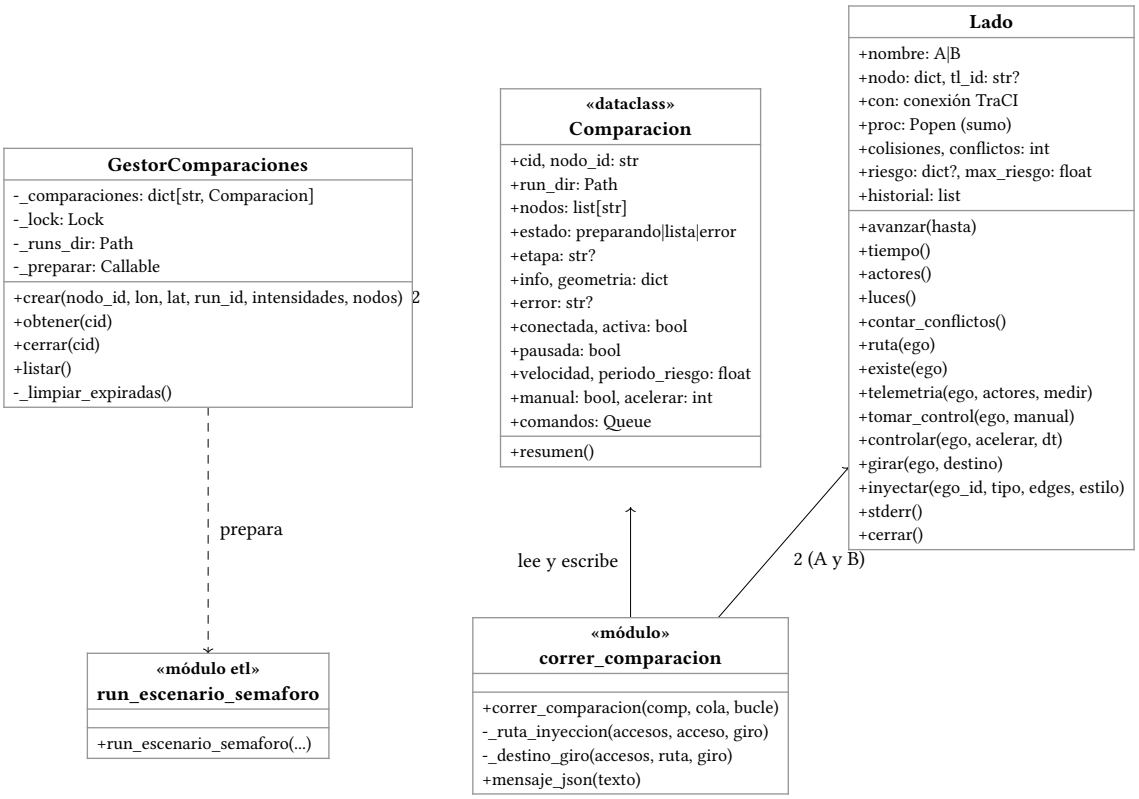


Figura 9: Diagrama de clases de la comparación 3D.

Tabla 19: Clases de la comparación 3D.

Clase	Responsabilidad, atributos y métodos
`GestorComparaciones`	Mantiene las comparaciones abiertas (máximo 2, si no `429`), prepara cada zona en un hilo y libera las abandonadas. **Métodos:** `crear` registra la comparación (`sem-AAAAMMDD-HHMMSS-<6 hex>`) y lanza el hilo de preparación, que al terminar la deja `lista` o `error`; `obtener` devuelve una comparación y renueva su `ultimo_ping`; `cerrar` la desactiva y borra su directorio en segundo plano (hasta 5 reintentos por el bloqueo de archivos de Windows); `listar` devuelve los resúmenes; `_limpiar_expiradas` elimina las que nadie conectó en 180 s. `_preparar` es inyectable en pruebas.
`Comparacion`	Estado de una comparación y mandos en vivo. `estado`, `etapa`, `info`, `geometria` y `error` describen la preparación; `conectada` impide que haya dos ventanas; `pausada`, `velocidad`, `periodo_riesgo`, `manual` y `acelerar` se leen en cada paso del bucle; `comandos` es la cola de los comandos que no son simples banderas (seguir, inyectar, manual, giro). `resumen` entrega el estado sin la geometría de accesos.
`Lado`	Una instancia de SUMO con TraCI y el estado que se le sigue por frame. El constructor lanza `sumo` con un puerto libre y se conecta. **Métodos:** `avanzar` lleva la simulación a un instante (filtra con `getIDList()` los actores que ya salieron antes de suscribirse); `tiempo` da el instante actual; `actores` devuelve vehículos y personas como `[id, tipo, x, y, z, ángulo, v, edge]`; `luces` el estado del semáforo (nulo en el lado A); `contar_conflictos` cuenta cruces con

Clase	Responsabilidad, atributos y métodos
	PET < 1,5 s; `ruta` y `existe` consultan al ego; `telemetria` arma la telemetría y el riesgo del ego; `tomar_control` y `controlar` implementan la conducción manual con `setSpeed`; `girar` cambia el destino con `changeTarget`; `inyectar` agrega un actor propio; `stderr` devuelve el final del *stderr* de SUMO; `cerrar` termina la conexión y el proceso.
`correr_comparacion`	Bucle síncrono (en un hilo) que avanza A y B a la par y publica frames en una `asyncio.Queue` (Sección 3.2.4). `_ruta_inyeccion` y `_destino_giro` calculan la ruta de un actor inyectado y el último *edge* de un giro; `mensaje_json` interpreta un mensaje de texto del cliente.
`run_escenario_semaforo`	Función del ETL que prepara `zona.net.xml`, `zona_tls.net.xml` y la demanda compartida (Sección 3.1.5).

Asociaciones. `GestorComparaciones` contiene hasta dos `Comparacion` y delega la preparación de la zona en `run_escenario_semaforo`. `correr_comparacion` lee y escribe una `Comparacion` (mandos y estado) y maneja exactamente dos `Lado`: A (red sin semáforo) y B (red con el semáforo propuesto).

3.1.3 ETL de demanda y simulación (`simulation-service/src/etl`)

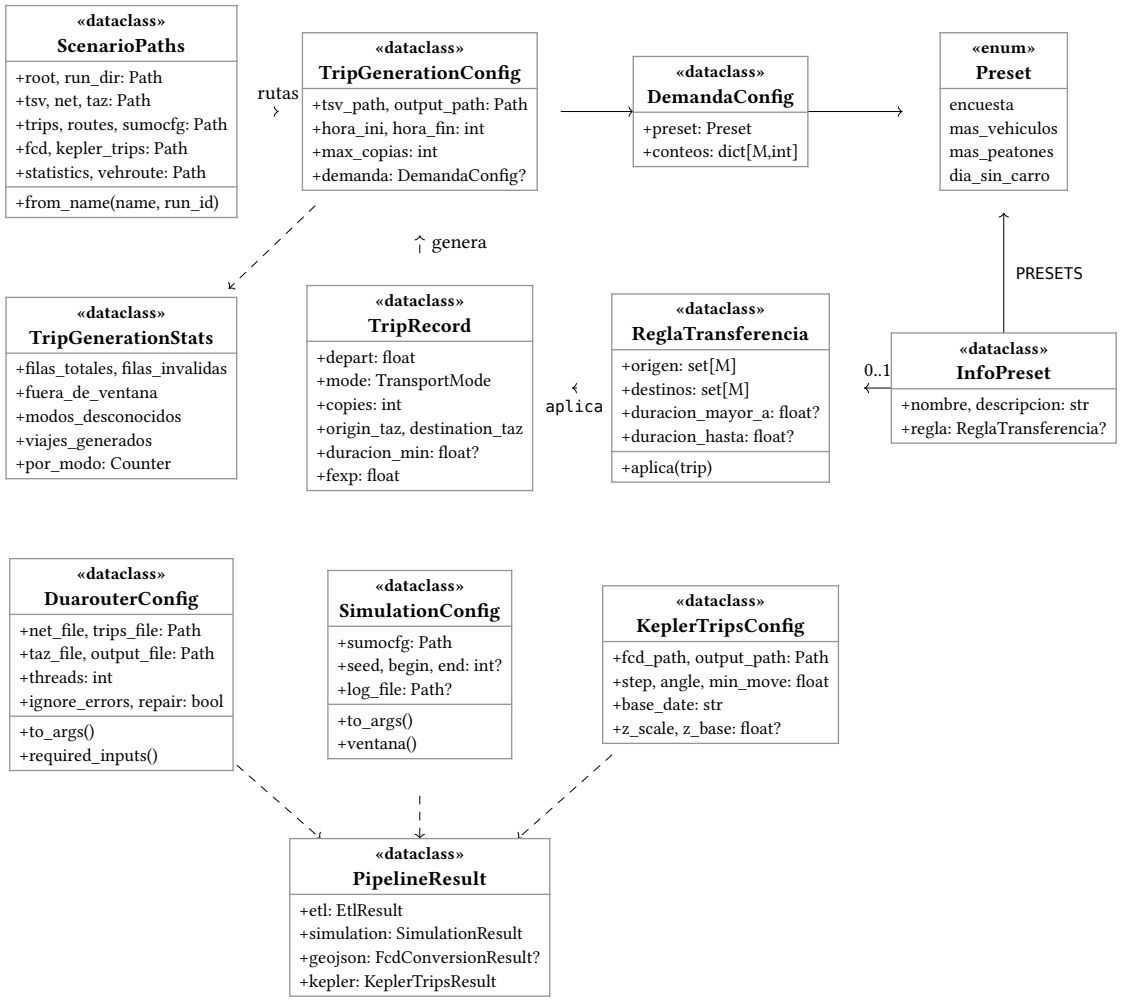


Figura 10: Diagrama de clases del ETL: configuraciones, demanda parametrizada y resultado del pipeline.

Tabla 20: Clases y módulos del ETL.

Clase o módulo	Responsabilidad, atributos y métodos
`ScenarioPaths`	Única fuente de verdad para las rutas de un escenario. Los insumos están en `scenarios/<nombre>/`; los archivos generados van ahí o, si se da `run_id`, en `runs/<run_id>/` para que las corridas no se pisen. `from_name(name, run_id)` arma todas las rutas; los nombres de salida deben coincidir con el `.sumocfg` (p. ej. `output-prefix="usaquen_am."`).
`TripGenerationConfig`	Parámetros de la generación de viajes: TSV de entrada, XML de salida, ventana horaria (07:00–10:00), tope de copias por viaje (`MAX_COPIAS` = 50) y la `demanda` parametrizada (nulo = encuesta tal cual).
`TripGenerationStats`	Contadores mutables: `filas_invalidas`, `fuera_de_ventana` y `modos_desconocidos` se cuentan, no se descartan en silencio; `viajes_generados` y `por_modo` resumen la salida. La clase `TripGenerationResult` agrupa el XML escrito y las estadísticas.
`TripRecord`	Un viaje de la encuesta ya validado, antes de expandirse en `copies` copias (`min(round(fexp), MAX_COPIAS)`).
`DemandaConfig`	Preset elegido y viajes por modo a generar.
`Preset`, `InfoPreset`	Los cuatro presets (`encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`); cada `InfoPreset` tiene nombre, descripción y una `ReglaTransferencia` opcional (`encuesta` no tiene).
`ReglaTransferencia`	Qué viajes cambian de modo (`origen`), hacia cuáles (`destinos`) y en qué rango de duración. `aplica(trip)` indica si un viaje cumple la regla.
`DuarouterConfig`	Parámetros de `duarouter`: red, viajes, TAZ y salida; `--ignore-errors` y `--repair`. `to_args()` arma la línea de comandos y `required_inputs()` lista los archivos que deben existir. El resultado (`DuarouterResult`) trae la salida estándar y los avisos de rutas no encontradas.
`SimulationConfig`	Parámetros de SUMO: `.sumocfg`, semilla y ventana. `to_args()` arma la línea de comandos y `ventana()` devuelve inicio y fin (los del config o los del `.sumocfg`). `SimulationResult` guarda la bitácora.
`KeplerTripsConfig`	Parámetros de la exportación a kepler.gl: FCD de entrada, salida, paso temporal (`step`), cambio mínimo de rumbo (`angle`), movimiento mínimo (`min_move`), fecha ancla y altura base. `KeplerTripsStats` cuenta entidades, puntos leídos y exportados y descartados; `KeplerTripsResult` agrupa la salida.
`PipelineResult`	Resultado completo de una corrida: `EtlResult` (viajes y rutas), `SimulationResult`, la conversión opcional a puntos y los trips de kepler.
`etl.etl` (módulo)	Orquesta las etapas: `run_etl` (viajes y `duarouter`), `simulate` (SUMO, con TraCI si hay `on_avance` o `cancelar`), `export_geojson`, `export_kepler_trips` y `run_pipeline` (las cuatro en secuencia, con cancelación entre etapas);

Clase o módulo	Responsabilidad, atributos y métodos
	`resumen_corrida` arma lo que se muestra de una corrida; `run_recorte` y `run_escenario_semaforo` preparan zonas.
`etl.precalculadas` (módulo)	`generar` simula un preset una vez y guarda `trips.geojson.gz` (`mtime=0`), `volumenes.json` y `meta.json`; `cargar`, `listar` y `leer_volumenes` los consultan; `huella_insumos` calcula la huella SHA-256 de TSV, red, TAZ y `.sumocfg` para detectar precalculadas obsoletas.
Módulos auxiliares	`trips` (lee el TSV, valida, expande, ordena por `depart` y escribe `*.trips.xml`), `demanda` (probabilidades P(modo | duración), reparto por mayor resto), `duarouter`, `simulation` (`run_simulation`, TraCI), `kepler` (`reduce_points`, `convert_fcd_to_kepler_trips`), `fcd`, `vtypes`, `estilos`, `cierres` (`CierreVial`, `escribir_rerouter`), `proceso` (`ejecutar`, `Cancelado`, `ultimo_depart`).

Tabla 21: Asociaciones del diagrama del ETL.

Asociación	Significado
`TripGenerationConfig` ⟶ `DemandaConfig`	La configuración de viajes incluye opcionalmente la demanda parametrizada.
`DemandaConfig` ⟶ `Preset`	Cada demanda nombra uno de los presets.
`Preset` ⟵ `InfoPreset` (`PRESETS`)	El diccionario `PRESETS` asocia cada preset con su información.
`InfoPreset` ⟶ `ReglaTransferencia` (0..1)	Un preset tiene a lo sumo una regla de transferencia modal.
`ReglaTransferencia` ⇢ `TripRecord`	`aplica` evalúa si un viaje cumple la regla.
`TripGenerationConfig` ⇢ `TripRecord`, `TripGenerationStats`	La generación produce registros de viaje y actualiza las estadísticas.
`ScenarioPaths` ⇢ `TripGenerationConfig`	Las rutas del escenario alimentan las configuraciones de cada etapa.
`SimulationConfig` ⇢ `PipelineResult`	El resultado de cada etapa (`*Result`) se agrega al `PipelineResult`.

3.1.4 Análisis de intersecciones (`etl/sumo`)

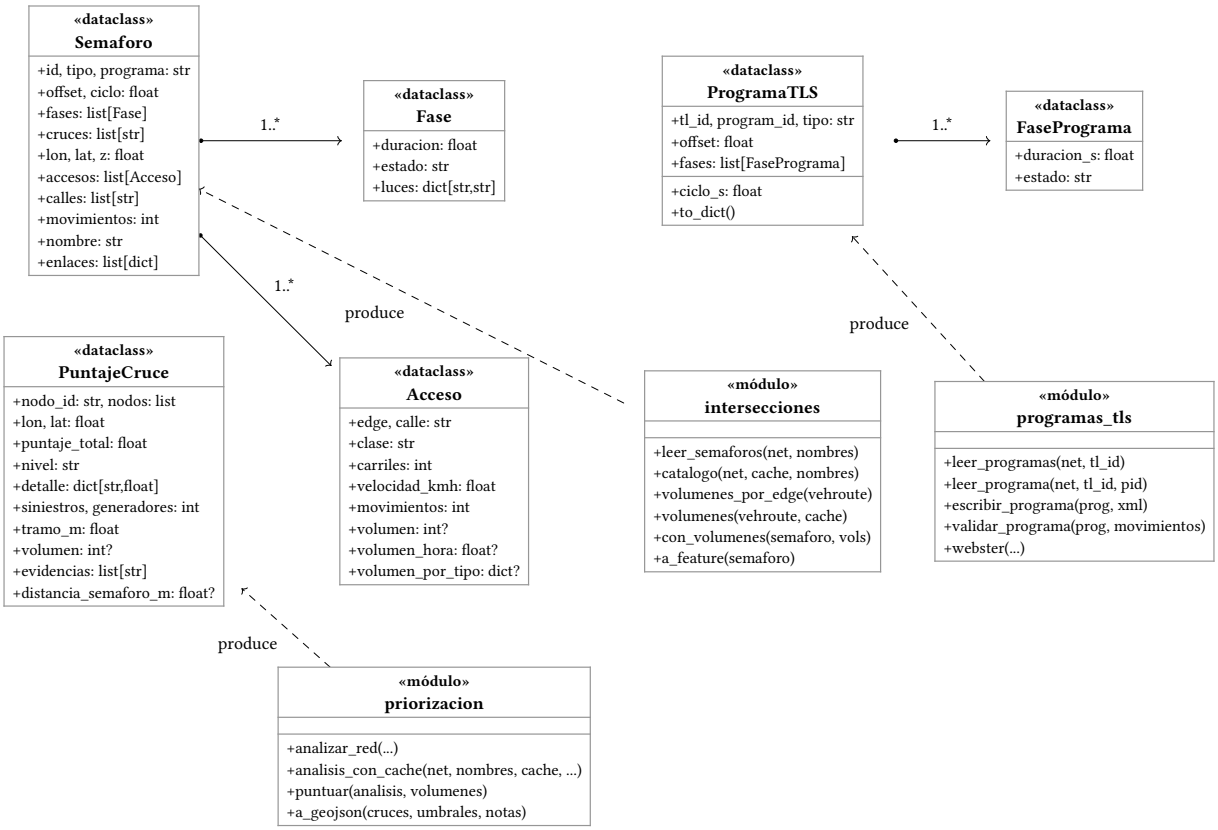


Figura 11: Diagrama de clases del análisis de intersecciones.

Tabla 22: Clases y módulos del análisis de intersecciones.

Clase o módulo	Responsabilidad, atributos y métodos
`Semaforo`	Un `tlLogic` de la red con su ubicación (lon/lat por `pyproj`, altura `z`), su programa activo, sus accesos y los enlaces `linkIndex → acceso y giro`. `cruces` son los nodos de SUMO que controla; `movimientos` el número de conexiones controladas.
`Fase`	Una fase del programa: duración, `estado` (una letra por `linkIndex`) y el color resultante por acceso (verde, amarillo, rojo, apagado).
`Acceso`	Una entrada al cruce: `edge`, nombre de la calle (de `nombres_vias.json`), clase de vía, carriles, velocidad y movimientos que controla; los volúmenes (`volumen`, `volumen_hora`, `volumen_por_tipo`) se llenan solo si hay una corrida con `vehroute`.
`ProgramaTLS`, `FasePrograma`	Un programa editable: `ciclo_s` (propiedad) es la suma de las duraciones y `to_dict()` lo serializa para la API.
`PuntajeCruce`	Un cruce sin semáforo evaluado: puntaje 0–100, nivel (`prioritario`, `recomendado`, `observacion`), el detalle de cinco componentes en 0–1 y la evidencia que lo respalda.
`intersecciones`	`leer_semaforos` extrae un `Semaforo` por `tlLogic` de la red; `catalogo` lo hace con caché (`cache/semaforos.json`, invalidada por mtime y tamaño); `volumenes_por_edge` cuenta, desde `vehroute.xml`, la última ruta de cada `routeDistribution`; `volumenes` lo hace con caché; `con_volumenes` copia un semáforo

Clase o módulo	Responsabilidad, atributos y métodos
	con los volúmenes de una corrida; `a_feature` lo convierte en un *feature* GeoJSON.
`programas_tls`	`leer_programas` y `leer_programa` leen los `tlLogic`; `escribir_programa` guarda un programa editado; `validar_programa` revisa que cada fase tenga un carácter por movimiento, un amarillo ≥ 3 s si hay verdes y un ciclo ≥ 15 s; `webster` calcula ciclo y verdes (Sección 3.2.5.6).
`priorizacion`	`analizar_red` agrupa nodos en cruces, aplica las reglas duras y reúne la evidencia (independiente de la corrida, con caché `priorizacion.json`); `analisis_con_cache` lo invalida por mtime, tamaño o `VERSION_ANALISIS`; `puntuar` calcula puntaje y niveles con los volúmenes de una corrida (Sección 3.2.5.5); `a_geojson` arma la respuesta de la API.

Asociaciones. Un `Semaforo` contiene al menos una `Fase` y un `Acceso`; un `ProgramaTLS` contiene al menos una `FasePrograma`. Los módulos `intersecciones`, `programas_tls` y `priorizacion` son fábricas: producen, respectivamente, `Semaforo`, `ProgramaTLS` y `PuntajeCruce` a partir de la red de SUMO, la evidencia versionada y los volúmenes de una corrida.

3.1.5 Semáforo propuesto, escena 3D y telemetría (`etl/sumo`)

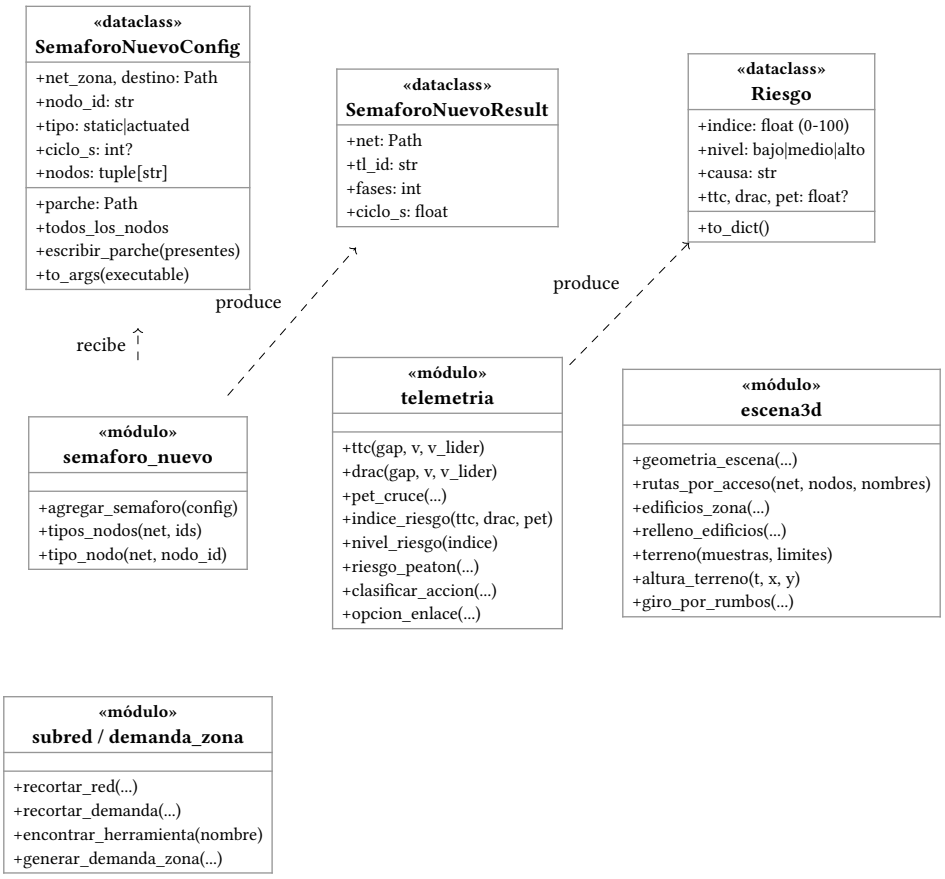


Figura 12: Diagrama de clases del semáforo propuesto, la escena 3D y la telemetría.

Tabla 23: Clases y módulos del semáforo propuesto, la escena 3D y la telemetría.

Clase o módulo	Responsabilidad, atributos y métodos
`SemaforoNuevoConfig`	Parámetros de la variante con semáforo: red de la zona sin semáforo, `nodo_id` (junction representativo y también id del `tlLogic`), `nodos` (todos los del cruce), tipo (`static` o `actuated`) y ciclo opcional. `escribir_parche` escribe `semaforo_propuesto.nod.xml` dando a **todos** los nodos presentes `type="traffic_light"` y el mismo `tl` (un solo `tlLogic` controla el cruce completo); `to_args` arma la línea de `netconvert` (`--tls.default-type`, `--geometry.remove false`); `parche` y `todos_los_nodos` son propiedades derivadas.
`SemaforoNuevoResult`	Red generada (`zona_tls.net.xml`), id del semáforo, número de fases y ciclo resultante. `SemaforoNuevoStats` cuenta las conexiones controladas.
`semaforo_nuevo`	`agregar_semaforo` verifica que ningún nodo del cruce tenga ya semáforo, escribe el parche, ejecuta `netconvert` y comprueba que se generó un programa con fases; `tipos_nodos` y `tipo_nodo` consultan el tipo de uno o varios *junctions*.
`Riesgo`	Resultado de evaluar un actor: índice 0–100, nivel, causa (el indicador que manda) y los valores de TTC, DRAC y PET. `to_dict` redondea para el frame.
`telemetria`	Funciones puras (Sección 3.2.5.8): `ttc`, `drac` y `pet_cruce` calculan los indicadores; `indice_riesgo` los combina (manda el mayor) y `nivel_riesgo` lo clasifica; `riesgo_peaton` evalúa a un peatón; `clasificar_accion` traduce velocidad, aceleración, luz, brecha y cesión de paso en una acción legible; `opcion_enlace` describe cada giro posible.
`escena3d`	`geometria_escena` produce el JSON de la escena una sola vez: calzadas, andenes, cebras y bocas de calle, un poste por acceso, terreno de rejilla IDW (`terreno`, `altura_terreno`) y edificios de OSM y Microsoft deduplicados (`edificios_zona`) más un relleno ilustrativo (`relleno_edificios`, `relleno: true`); `rutas_por_acceso` calcula las rutas de entrada y salida de cada acceso; `giro_por_rumbos` clasifica un giro.
`subred`, `demanda_zona`	Preparan la zona: `recortar_red` (`netconvert`) y `recortar_demanda` (`cutRoutes.py`) alrededor de un punto; `generar_demanda_zona` suma viajes reales recortados y tráfico de fondo por modo (`randomTrips.py`); `encontrar_herramienta` localiza los scripts de SUMO.

Asociaciones. `semaforo_nuevo` recibe un `SemaforoNuevoConfig` y produce un `SemaforoNuevoResult`; `telemetria` produce `Riesgo`. `run_escenario_semaforo` (Sección 3.1.3) encadena `subred`, `demanda_zona`, `semaforo_nuevo` y `escena3d` para dejar la zona lista, y `Lado` (Sección 3.1.2) usa `telemetria` en cada frame.

3.2 Comportamiento del sistema

Esta sección describe cómo cambia el sistema en el tiempo. Para cada acción compleja de la Sección 2.3 hay un diagrama de secuencia que usa las instancias y los métodos de las clases de la Sección 3.1; no se describe lo que sucede dentro de las librerías de terceros (FastAPI, TraCI, SUMO). Las acciones con pocas llamadas (por ejemplo, consultar una intersección o cargar una precalculada, que solo devuelve un archivo) no se diagraman. Al final se explican los algoritmos más complejos como guía de implementación.

3.2.1 Simulación parametrizada

Corresponde a los pasos 4 a 8 del proceso P1 (Figura 3). La API responde enseguida y la corrida avanza en un hilo aparte mientras el visor consulta su estado.

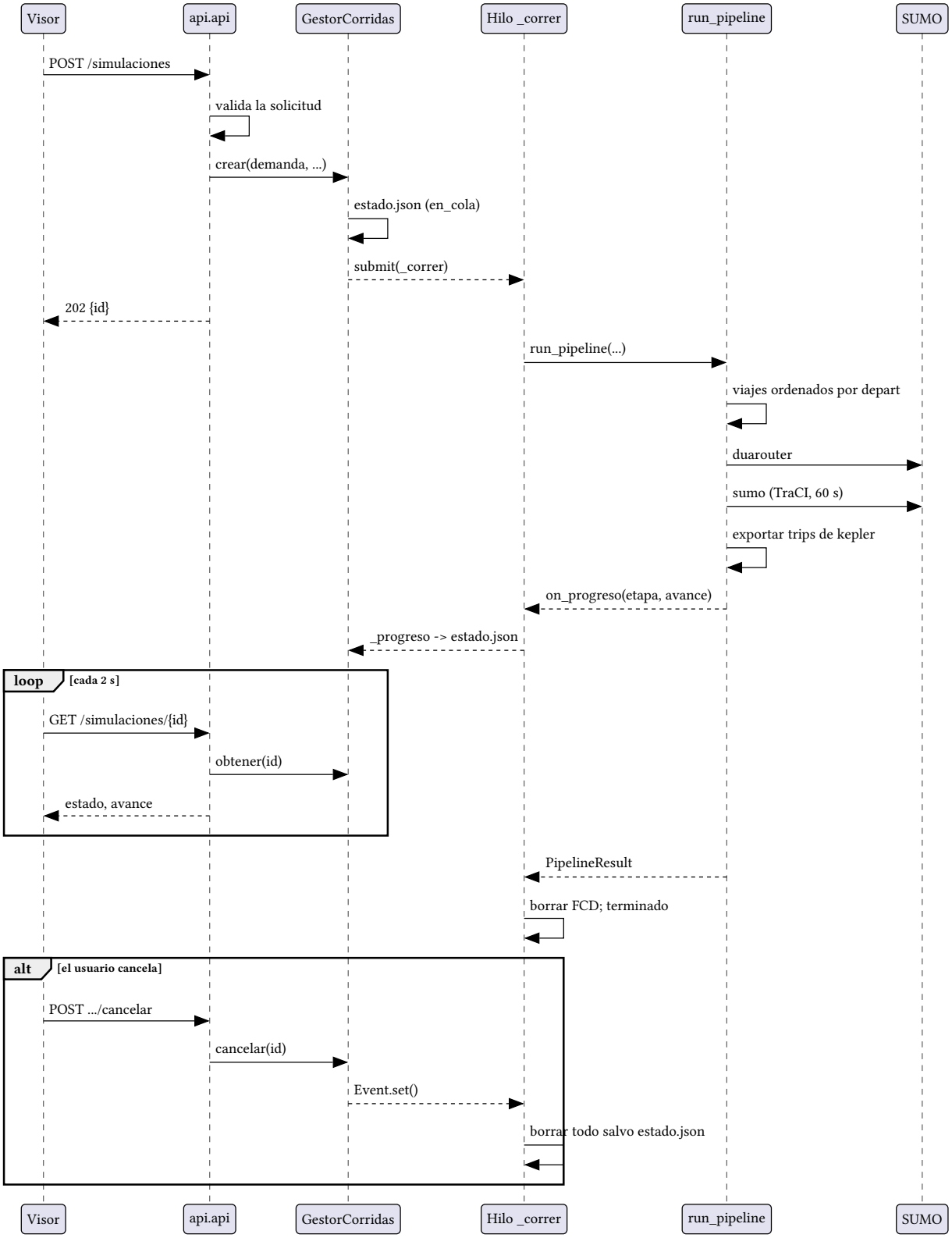


Figura 13: Diagrama de secuencia de una simulación parametrizada.

Tabla 24: Mensajes relevantes de la simulación parametrizada.

Mensaje	Explicación
`POST /simulaciones`	La API valida el cuerpo (`SolicitudSimulacion`): total de viajes entre 1 y 100.000; si falla responde `422` sin crear nada.
`crear(...)`	`GestorCorridas.crear` genera el `id`, guarda `runs/<id>/estado.json` en estado `en_cola` y envía `_correr` al `ThreadPoolExecutor(1)`; solo corre una a la vez y las demás esperan en cola. La API devuelve `202` con el estado inicial.
`run_pipeline`	Genera los viajes (ordenados por hora de salida), llama a `duarouter`, simula con SUMO bajo TraCI y exporta los trips de kepler. Entre etapas revisa la cancelación (`revisar_cancelacion`).
`on_progreso(etapa, avance)`	Cada etapa informa su avance; `_progreso` lo mapea a la franja global (`FRANJAS_ETAPA`: demanda 0–2 %, ruteo 2–15 %, simulación 15–85 %, exportación 85–100 %) y `_guardar` escribe `estado.json` como mucho cada 2 s.
`GET /simulaciones/{id}`	El visor consulta cada 2 s hasta que el estado sea final (Figura 14).
`cancelar(id)`	Activa el `threading.Event` de la corrida; este llega hasta el subproceso, que se termina. El trabajador lanza `Cancelado` y borra todos los archivos de la corrida menos `estado.json`.
Al terminar	`_correr` elimina el FCD (muy pesado, 1,4 GB con la demanda completa) y marca la corrida `terminado` con el resumen (`resumen_corrida`).

3.2.2 Ciclo de vida de una corrida

El atributo `estado` de `Corrida` evoluciona así:

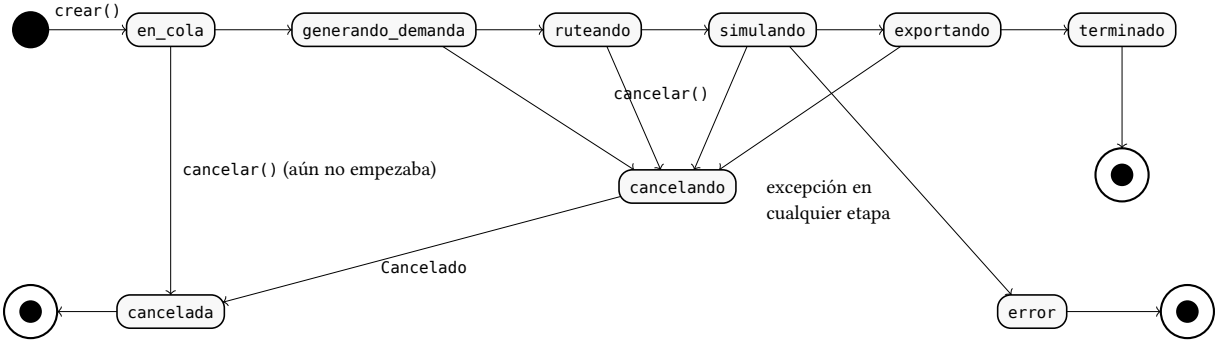


Figura 14: Diagrama de estados de una corrida.

Desde cualquier etapa activa (`generando_demanda`, `ruteando`, `simulando`, `exportando`) se puede pasar a `cancelando` con `cancelar()` o a `error` si una etapa lanza una excepción (se guarda el mensaje, últimos 2.000 caracteres). Tras un reinicio del servicio, las corridas que no estaban en un estado final (`terminado`, `error`, `cancelada`) pasan a `error` con el mensaje «Interrumpida: el servicio se reinició».

3.2.3 Preparación de la comparación 3D

Corresponde a los pasos 4 a 8 del proceso P3 (Figura 5): la API responde enseguida y la zona se prepara en un hilo aparte, lo que puede tardar decenas de segundos.

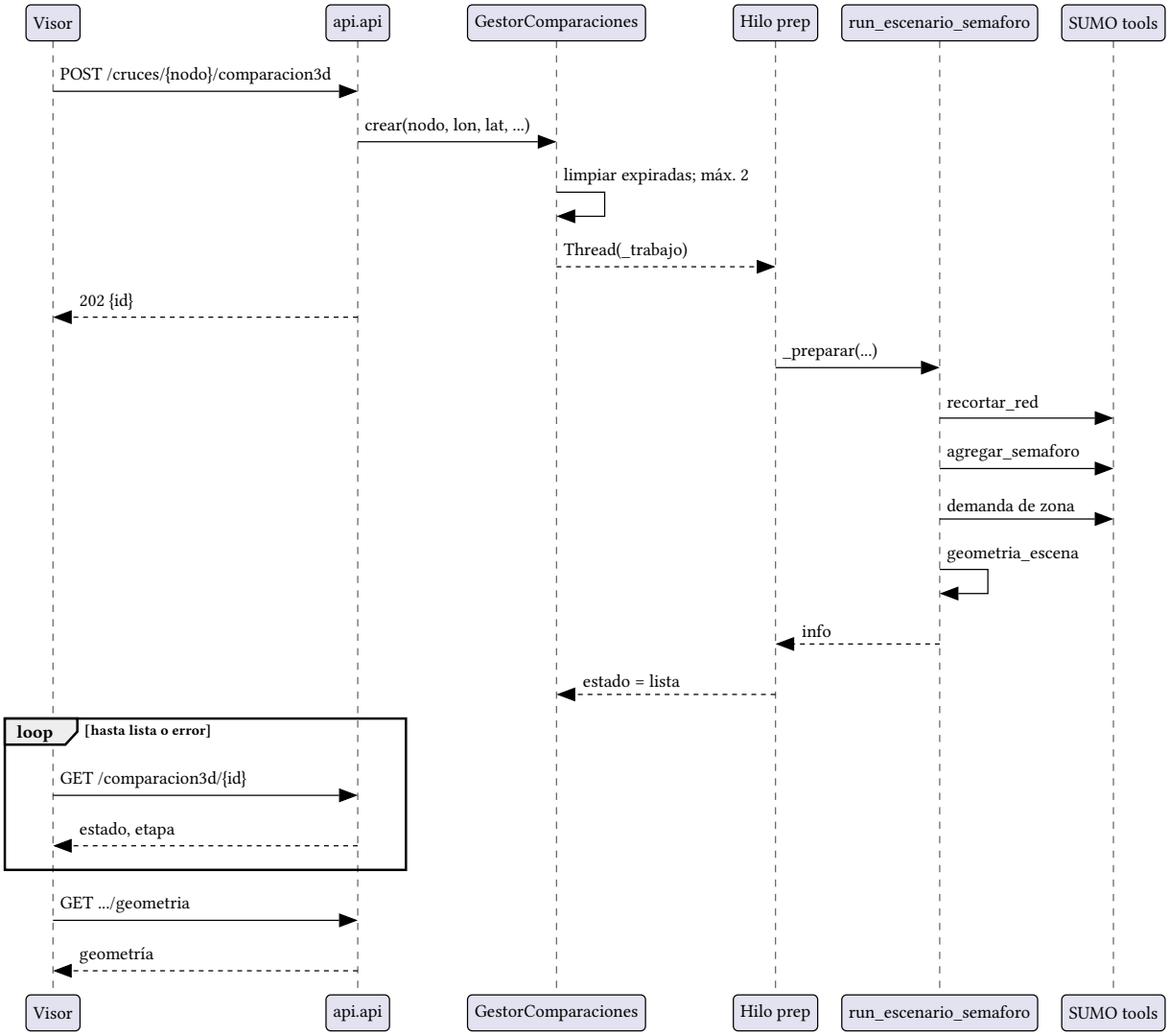


Figura 15: Diagrama de secuencia de la preparación de una comparación 3D.

Tabla 25: Mensajes relevantes de la preparación.

Mensaje	**Explicación**
`crear(...)`	Registra la `Comparacion` en estado `preparando`; si ya hay dos abiertas lanza un error que la API traduce a `429`. Lanza un hilo de preparación y devuelve enseguida.
`recortar_red`	Recorta la red de Usaquén alrededor del cruce (radio de 350 m) con `netconvert`: `zona.net.xml` es el lado A (tal como está).
`agregar_semaforo`	Crea el lado B: parche que da `type="traffic_light"` y el mismo `tl` a todos los nodos del cruce, y `netconvert` genera `zona_tls.net.xml` (Sección 3.2.5.7).
Demanda de zona	Viajes reales de la corrida pedida (o la más reciente con `vehroute`) recortados con `cutRoutes.py`, más tráfico de fondo por modo con `randomTrips.py`; la demanda es idéntica en los dos lados.
`geometria_escena`	Genera una sola vez el JSON de la escena (calzadas, andenes, terreno, edificios) que el visor convierte en geometría three.js.
Si algo falla	El hilo deja la comparación en `error` con el mensaje (máx. 500 caracteres) y el visor lo muestra.

3.2.4 Ejecución en vivo de la comparación 3D

Corresponde a los pasos 8 a 12 del proceso P3. Al conectarse el WebSocket se arranca un hilo que lanza los dos SUMO, los calienta 240 s simulados y emite un frame por cada paso de 0,2 s simulados.

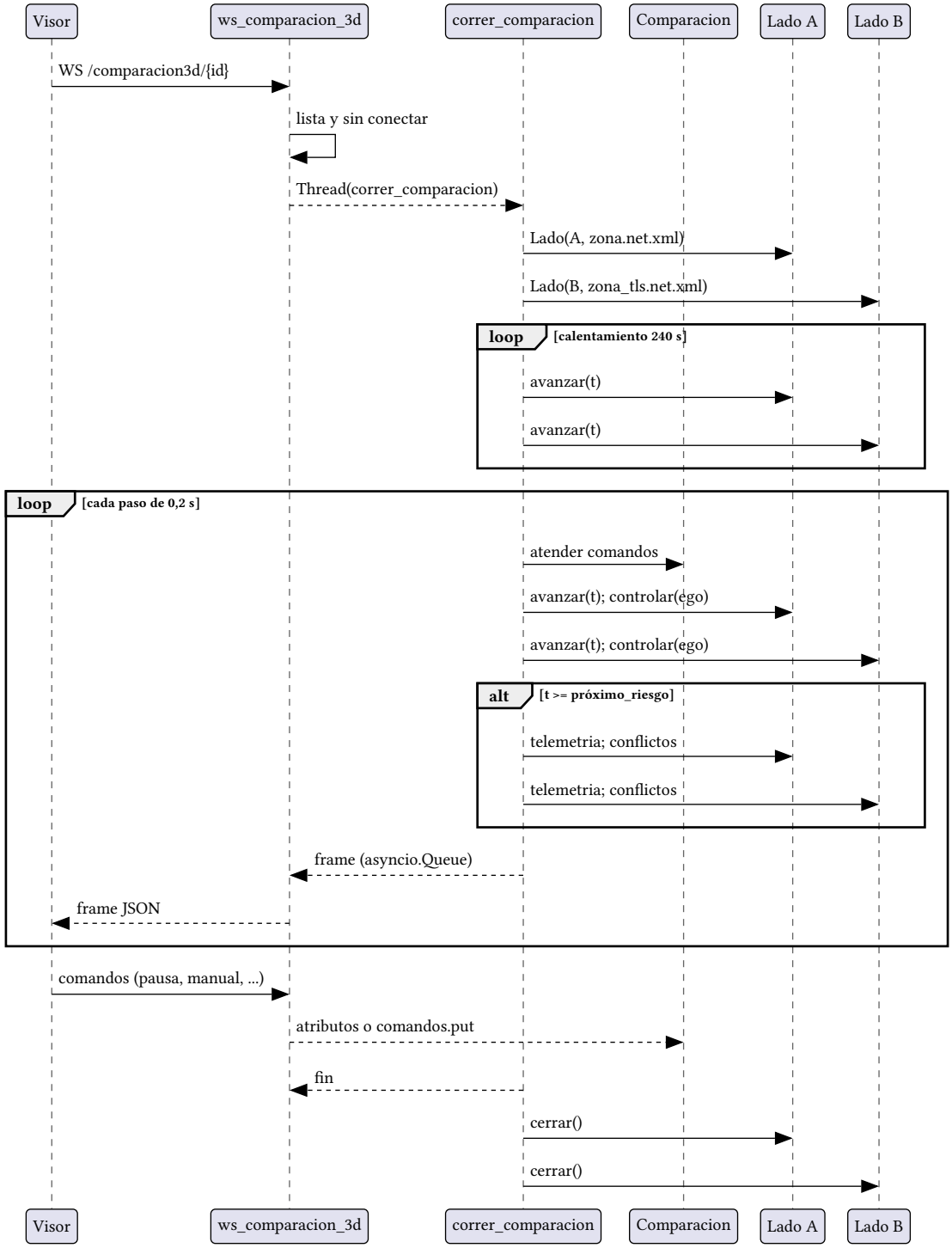


Figura 16: Diagrama de secuencia de la ejecución en vivo de la comparación 3D.

El bucle de `correr_comparacion` es el siguiente:

correr_comparacion(comp, cola, bucle):

1. iniciar lado A (`zona.net.xml`) y lado B (`zona_tls.net.xml`) con la misma demanda y semilla 42
2. **mientras** t < t_0 + 240 s: avanzar ambos de 20 en 20 s (calentamiento)
3. **mientras** comp.activa y t < fin:
   1. atender comandos (`manual`, `control`, `giro`, `seguir`, `inyectar`)
   2. si está en pausa: dormir 0,1 s y continuar
   3. pasos ⟵ max(1, round(velocidad)); t ⟵ t + pasos · 0,2 s
   4. **para cada lado**: avanzar a t; si manual, aplicar el mismo control al ego
   5. si t ≥ próximo_riesgo: medir riesgo y contar conflictos
   6. armar el frame (actores, luces, métricas, telemetría del ego) y publicarlo
   7. dormir lo necesario para mantener el ritmo de tiempo real
4. publicar `fin`; cerrar ambos lados

Listado 1: Pseudocódigo del bucle de la comparación 3D.

Puntos de diseño del bucle:

- **Equidad de la comparación.** Misma demanda, misma semilla y mismo conjunto de comandos aplicados a los dos lados. El ego se inyecta con el mismo id y la misma ruta en ambos (`_ruta_inyeccion`); los peatones usan `findIntermodalRoute` entre *edges* caminables.
- **Contrapresión.** La cola de frames tiene tamaño 8 y, si se llena, descarta el más antiguo: es preferible perder un frame que acumular retraso.
- **Suscripciones TraCI seguras.** Nunca se suscribe a un actor que ya salió de la red: SUMO 1.27 concatena los errores de suscripciones fallidas y falla (`Storage::writeUnsignedByte(): Invalid value`) cuando el mensaje pasa de 255 bytes. `Lado.avanzar` filtra los ids con `getIDList()`. Los vehículos inyectados que aún esperan entrar tienen `road id` vacío y se omiten.
- **Modo manual.** Acelerar y frenar se traducen a `setSpeed` (+1,8 m/s² y −3,5 m/s²), de modo que SUMO conserva sus chequeos de seguridad (no atraviesa un rojo ni choca). Los giros usan `changeTarget` hacia el último *edge* de la ruta del giro elegido.
- **Cierre.** Al cerrar la ventana el directorio `runs/sem-<id>/` se borra; si nadie se conecta en 180 s, la comparación expira.

3.2.5 Algoritmos de apoyo

Los siguientes algoritmos son la guía para implementar los métodos más complejos de las clases de la Sección 3.1.

3.2.5.1 Generación de viajes y orden de salida

Para cada fila válida del TSV se crea un `TripRecord` con `copies = min(round(fexp), MAX_COPIAS)`. Las copias se emiten con ids únicos y **se ordenan por `depart`** antes de escribir el XML. El orden es indispensable: el TSV no está ordenado, `duarouter` conserva el orden de entrada y SUMO **descarta** (no reordena) cualquier entidad cuya salida sea menor que la máxima ya cargada. Sin el orden se perdían cerca de 23.000 de 24.600 viajes. Los vehículos se escriben como `<trip>`; los peatones y el transporte público, como `<person><walk>`.

3.2.5.2 Demanda parametrizada por presets

Un preset es una regla de transferencia modal `ReglaTransferencia(origen, destinos, duracion_mayor_a, duracion_hasta)`. Con los viajes de **todo el día** se calcula $P(m \mid r)$: la probabilidad de cada modo m en cada rango de duración r (≤ 15, 15–30, 30–60 y > 60 min), ponderada por `fexp`. Después:

1. Para cada viaje de la ventana que cumple la regla, su peso (`copies`) se reparte entre los modos destino en proporción a $P(m \mid r)$ restringida a esos destinos; los demás viajes conservan su modo.
2. Resultan *pools* por modo con pares (viaje, peso).
3. Para cada modo con conteo pedido n_m, se asignan copias a los viajes del *pool* con el método del **mayor resto** (Hamilton): $n_m \frac{w_i}{\sum_j} w_j$ se trunca y las unidades que faltan se dan a las partes decimales mayores. La suma es exacta y el resultado es determinista.
4. Si un modo no tiene *pool* en el preset (p. ej. autos en «día sin carro»), se usa el *pool* base de la encuesta.

Propiedades verificadas por pruebas: los presets conservan el total; un viaje transferido mantiene su origen/destino y su hora de salida; y con `encuesta` y sus conteos por defecto se reproduce exactamente la expansión simple de la encuesta.

3.2.5.3 Ejecución de SUMO bajo TraCI

`simulation._run_traci` lanza `sumo` con `--remote-port` libre, se conecta con reintentos y avanza con `simulationStep` en saltos de 60 s hasta `<end>`. Tras cada salto calcula el avance de la ventana y revisa la cancelación; ante cancelación o tiempo máximo mata el proceso. Los archivos de salida son los mismos que sin TraCI. Se usa TraCI, y no el registro de SUMO, porque con la salida redirigida a un *pipe* SUMO y `duarouter` almacenan su bitácora por bloques y esta solo llega al terminar; por eso el avance de `duarouter` se mide con el último `depart` escrito en el `.rou.xml`.

3.2.5.4 Reducción de puntos para kepler

`kepler.reduce_points` primero colapsa cada tramo quieto (misma posición en puntos consecutivos) a su primer y último punto y luego muestrea lo que queda: siempre conserva el primer y el último punto de la entidad, y entre ellos guarda un punto cada `step` segundos (8 s en el pipeline; 5 s por defecto del módulo) o cuando el rumbo cambia al menos 20°. Las entidades que se mueven menos de `min_move` = 20 m se descartan y se cuentan. Las posiciones con `inf` o `z` fuera de ±100.000 se ignoran como inválidas.

3.2.5.5 Priorización de cruces sin semáforo

El análisis de la red (independiente de la corrida) y el puntaje (dependiente de los volúmenes) se separan para poder cachear la parte costosa.

Análisis (con caché `priorizacion.json`):

1. Se agrupan nodos de SUMO a menos de 20 m (extensión máxima 45 m) y se fusionan los grupos del mismo par de calles a menos de 150 m (las dos calzadas de una avenida; extensión máxima 200 m). Un cruce requiere dos calles distintas que se corten a más de 35°.
2. **Reglas duras, todas obligatorias:** participa una vía colectora (`tertiary`) o superior, sin autopistas ni troncales; la otra vía es al menos residencial; sin rampas ni glorietas a menos de 40 m; conflicto real (la vía secundaria cruza de frente o gira a la izquierda); sin semáforo a menos de 80 m.
3. **Evidencia**, calibrada contra los cruces ya semaforizados de la propia red (*backtest*): siniestros a menos de 30 m ≥ p75 (mínimo 3), o ≥ mediana con un generador cerca; tramo de la vía principal sin semáforo ≥ 600 m; volumen simulado de entrada ≥ p75. Generadores: colegios, jardines, universidades, hospitales y clínicas a menos de 200 m.

Puntaje (en cada consulta): número de evidencias «fuertes» $e \longrightarrow$ nivel `prioritario` ($e \geq 2$), `recomendado` ($e = 1$) u `observacion` (solo generador o siniestros ≥ mediana). El puntaje es un piso por nivel (75 / 50 / 25) más hasta 25 puntos:

$$\text{puntaje} = \text{piso} + 25(0,35s + 0,30v + 0,15g + 0,10t + 0,10h)$$

con s el rango percentil de siniestros, v el de volumen, $g = \min\left(\frac{g_n}{3}, 1\right)$ los generadores, $t = \min\left(\frac{\text{tramo}}{1200}, 1\right)$ y h la jerarquía. Finalmente, una *supresión de no máximos* (NMS): dos cruces del mismo

nivel a menos de 120 m se estudian juntos y el de menor puntaje baja un nivel. La lógica sigue la del proyecto Semáforos de Barranquilla, sin su modelo de lenguaje.

3.2.5.6 Programas de semáforo y Webster

`validar_programa` exige que cada fase tenga tantos caracteres como movimientos controlados, que haya al menos una fase amarilla ≥ 3 s si hay verdes y un ciclo ≥ 15 s. El cálculo de Webster [8] usa:

$$C_0 = \frac{1{,}5L + 5}{1 - Y}, \quad L = n_{\text{fases}} \cdot 3 \text{ s}, \quad Y = \sum_i \max_j \left(\frac{q_j}{s} \right)$$

con flujo de saturación s = 1.800 veh/h por carril. Los verdes se reparten en proporción a $\frac{y_i}{Y}$ (mínimo 5 s) y el ciclo mínimo es 30 s; si $Y \geq 1$ la intersección está sobresaturada y se devuelve un error. **Limitación conocida:** los accesos se asignan a las fases de forma circular (no según los movimientos compatibles reales), y si no hay volumen simulado se usa un valor de demostración de 300 veh/h. La API lo advierte en la respuesta (`nota`).

3.2.5.7 Variante con semáforo propuesto

`agregar_semaforo` verifica que ningún nodo del cruce tenga ya semáforo, escribe un parche `semaforo_propuesto.nod.xml` que da a **todos** los nodos del cruce `type="traffic_light"` y el mismo `tl` (un solo `tlLogic` controla el cruce completo, p. ej. las dos calzadas) y ejecuta `netconvert` con `--tls.default-type` sobre la red de la zona (`--geometry.remove false`). Luego comprueba que se generó un programa con fases.

3.2.5.8 Indicadores de riesgo

Funciones puras en `telemetria.py`, calculadas sobre lo que TraCI reporta del actor seguido:

- $\text{TTC} = \frac{\text{gap}}{v - v_{\text{líder}}}$ si el actor se acerca al líder.
- $\text{DRAC} = \frac{(v - v_{\text{líder}})^2}{2 \cdot \text{gap}}$ (m/s²).
- PET aproximado en el cruce: diferencia entre las llegadas del ego y de un vehículo en conflicto (`getJunctionFoes`) al punto de cruce.

Cada indicador se normaliza de 0 (umbral seguro) a 100 (umbral crítico o peor): TTC 6 ⟶ 1,5 s; DRAC 0 ⟶ 3,35 m/s²; PET 5 ⟶ 1,5 s. El índice de riesgo es el **mayor** de los tres (manda el más grave, que además se informa como causa) y su nivel es bajo (< 30), medio (30–59) o alto (≥ 60); los umbrales provienen de la literatura de SSM [9]. `clasificar_accion` traduce velocidad, aceleración, luz próxima, brecha y cesión de paso en una acción legible («Detenido en el semáforo», «Frenada fuerte», etc.). Las métricas por lado cuentan como conflicto un cruce con PET < 1,5 s entre vehículos a menos de 60 m del nodo.

3.2.5.9 Escena 3D

`geometria_escena` produce, una sola vez, el JSON que el visor convierte en geometría: calzadas, andenes, cebras y bocas de calle desde la red; un poste de semáforo por acceso; un terreno de rejilla por interpolación de distancia inversa (IDW) a partir de las alturas de calzadas y cruces; edificios de OSM y Microsoft deduplicados por solape, más un relleno ilustrativo (`relleno: true`) en los frentes de manzana vacíos que no se superponga con los reales. En el cliente, las coordenadas de SUMO se transforman como $(x, z - z_0, -(y - y_0))$ con el origen en el nodo, y las mallas estáticas se funden por material para reducir llamadas de dibujo.

3.3 Persistencia

El sistema **no usa un motor de base de datos**. Los datos se guardan como archivos (TSV, XML de SUMO, JSON y GeoJSON) organizados por escenario y por corrida, y las estructuras en memoria son `dataclass` de Python. Se decidió así porque el volumen es pequeño (un único escenario), los resultados

de SUMO ya son archivos y se evita instalar y operar un servicio adicional. Por eso, en lugar de un diagrama entidad–relación sobre tablas, el modelo se describe con un diagrama de clases con estereotipos [11] en el que cada «directorio» y cada «archivo» juega el papel de una entidad, y su ubicación en disco el de su esquema.

3.3.1 Modelo de persistencia

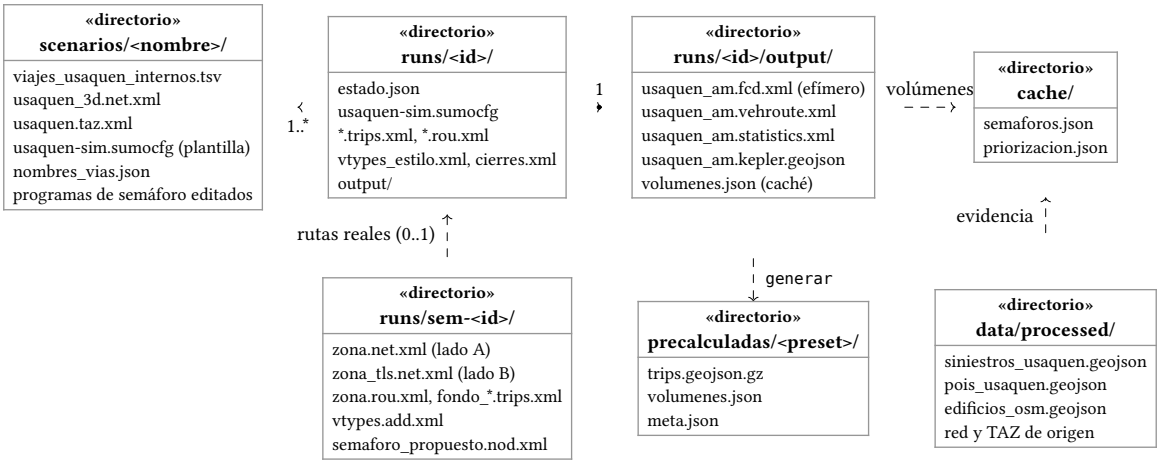


Figura 17: Diagrama de clases con estereotipos del modelo de persistencia.

Tabla 26: Elementos del modelo de persistencia.

Elemento	Descripción
`scenarios/<nombre>/`	Insumos de un escenario (hoy solo `default`). El TSV de viajes internos (2.580 filas) es el producto de los cuadernos de exploración, ya filtrado a viajes con origen y destino en Usaquén y con las zonas ZAT mapeadas a TAZ. La red 3D y las zonas también se generan fuera del servicio y se copian aquí.
`runs/<id>/`	Una corrida lanzada desde la API: sus archivos intermedios (viajes, rutas, `.sumocfg`) y su `estado.json` (Sección 3.3.2.2). Cada corrida tiene sus propias salidas en `output/` y se crea a partir de la plantilla del escenario, de modo que varias corridas no se pisan.
`runs/<id>/output/`	Salidas de SUMO (FCD, `vehroute`, estadísticas) y la exportación para kepler. El FCD se borra tras exportar. `volumenes.json` es la caché de volúmenes por *edge* de esa corrida.
`precalculadas/<preset>/`	Un preset ya simulado y versionado: lo único que necesita el visor (Sección 3.3.2.4).
`runs/sem-<id>/`	Una comparación 3D: la red recortada sin y con el semáforo propuesto, la demanda compartida y el parche de nodos. Se borra al cerrar la comparación.
`cache/`	Caché del análisis de red independiente de la corrida: catálogo de semáforos y análisis de priorización.
`data/processed/`	Evidencia descargada una sola vez (siniestros SDM, POI y edificios OSM/Microsoft) y artefactos de red de los cuadernos.

Asociaciones. Un escenario da origen a muchas corridas (1..*); cada corrida contiene exactamente un directorio de salidas (composición). Una comparación 3D puede tomar rutas reales de una corrida (0..1). Una precalculada se genera a partir de las salidas de una corrida del escenario. Las cachés se derivan de la red del escenario, de los volúmenes de una corrida y de la evidencia de `data/processed/`.

3.3.2 Diccionario de datos

3.3.2.1 Viaje de la encuesta (TSV de entrada)

Tabla 27: Columnas del TSV de viajes y su uso.

Campo	Tipo	Descripción y restricciones
`zat_ori`	entero	Zona de origen; se mapea a un id de TAZ. Fila inválida si falta.
`zat_des`	entero	Zona de destino. Fila inválida si falta.
`hora_ini_seg`	real	Hora de salida en segundos desde las 00:00. Solo se aceptan viajes entre `HORA_INI` (25.200) y `HORA_FIN` (36.000).
`hora_ini_hhmmss`	texto	Misma hora en formato legible (informativo).
`duracion_min`	real, opcional	Duración reportada; define el rango de duración para los presets.
`modo_principal_agrupado`	texto	Etiqueta cruda (`SurveyMode`, 11 valores) que se convierte en `TransportMode` (10 modos).
`motivo_viaje`	texto	Motivo del viaje (informativo, no usado por el pipeline).
`fexp_vj`	real	Factor de expansión. El viaje se replica `round(fexp_vj)` veces, con tope `MAX_COPIAS` = 50.

Las filas inválidas, fuera de ventana o con modo desconocido no se descartan en silencio: se cuentan en `TripGenerationStats` y se reportan en el resumen de la corrida.

3.3.2.2 Estado de una corrida (`runs/<id>/estado.json`)

Tabla 28: Campos de `estado.json` (dataclass `Corrida`).

Campo	Tipo	Descripción
`id`	texto	`AAAAMMDD-HHMMSS-<6 hex>`; clave primaria de la corrida y nombre del directorio.
`preset`	texto	Uno de `encuesta`, `mas_vehiculos`, `mas_peatones`, `dia_sin_carro`.
`conteos`	mapa modo ⟶ entero	Viajes pedidos por modo; la suma debe estar entre 1 y 100.000.
`semilla`	entero o nulo	Semilla de SUMO; nulo = la del `.sumocfg`.
`estilo`	texto	`agresivo`, `normal` o `cauteloso`.
`cierres`	lista	Cierres viales: `edge_id`, `hora_inicio_s`, `hora_fin_s`.
`estado`	texto	Ver Figura 14.
`avance`	real 0–1	Avance global (según `FRANJAS_ETAPA`).
`avance_etapa`	real o nulo	Avance dentro de la etapa; nulo si no se puede medir.
`creada`, `iniciada`, `actualizada`	ISO 8601	Marcas de tiempo.
`error`	texto o nulo	Mensaje si falló.

Campo	Tipo	Descripción
`resultado`	objeto o nulo	`viajes_generados`, `viajes_por_modo`, `trips_kepler` y las estadísticas de SUMO (`vehiculos_insertados`, `teleports`, `colisiones`, …).

3.3.2.3 Trips para kepler (GeoJSON de `LineString`)

Tabla 29: Esquema de cada *feature* de `kepler.geojson`.

Campo	Tipo	Descripción
`properties.id`	texto	Id del vehículo o persona. Para personas, el prefijo (`peaton_N`, `transporte_publico_N`) determina el modo.
`properties.mode`	texto	`TransportMode`; el visor le asigna color.
`properties.category`	texto	`vehiculo` o `persona`.
`properties.inicio_s`, `fin_s`	entero	Segundos de simulación al aparecer y al salir.
`properties.dist_m`, `avg_kmh`	numérico	Distancia recorrida y velocidad media.
`geometry.coordinates[i]`	`[lon, lat, z, t]`	`z` relativa al mínimo del FCD; `t` = *unix time* = 2000-01-01 UTC + segundos de simulación (07:00 en pantalla = 25.200 s en SUMO).

3.3.2.4 Metadatos de una precalculada (`meta.json`) y volúmenes

`meta.json` contiene `preset`, `conteos`, `semilla`, `generada`, `tamano_original`, `tamano_gz`, `resultado` (el mismo resumen de una corrida) e `insumos`: una huella SHA-256 (16 primeros hex) de TSV, red, TAZ y `.sumocfg`, con los saltos de línea normalizados. Sirve para detectar que una precalculada quedó desactualizada respecto a sus insumos.

`volumenes.json` mapea cada `edge` a un contador por modo (por ejemplo `{"-1012541443#0": {"bici": 12}}`). Se obtiene recorriendo `vehroute.xml` y contando la última ruta de cada `routeDistribution`; reemplaza al `vehroute` (22 MB) en las precalculadas.

3.3.2.5 Cachés de intersecciones

`cache/semaforos.json` guarda el catálogo de `Semaforo` (Sección 3.1.4) y `cache/priorizacion.json` el análisis de cruces sin semáforo con sus puntajes base. Ambos guardan la firma (mtime y tamaño) de sus insumos (la red y, en `priorizacion.json`, también la evidencia y los nombres de vías) y se descartan si cambia; `priorizacion.json` también si cambia `VERSION_ANALISIS`. El puntaje que depende de los volúmenes de una corrida no se guarda: se recalcula en cada consulta porque es barato.

3.3.2.6 Mensaje de *frame* del simulador 3D

Aunque no se persiste, es la estructura de datos más grande que circula en tiempo real. Cada frame del WebSocket es un objeto JSON `{tipo: "frame", t, dt, ego, manual, lados: {a: L, b: L}}`, donde cada lado `L` contiene:

- `vehiculos` y `personas`: listas de actores `[id, tipo, x, y, z, ángulo, velocidad, edge]` en coordenadas de la red.
- `luces`: `{estado, fase, restante}` del semáforo (nulo en el lado A, que no tiene).
- `metricas`: `vehiculos`, `peatones`, `en_cruce`, `detenidos_cruce`, `velocidad_media_cruce_kmh`, `espera_media_cruce_s`, `espera_lateral_s`, `colisiones`, `conflictos`, `riesgo_max`.

- `ego`: telemetría del actor seguido (velocidad, aceleración, líder, riesgo `TTC/DRAC/PET`, acción, decisiones posibles en el próximo cruce).

3.3.3 Estrategia de almacenamiento

Tabla 30: Ubicaciones de almacenamiento y su política.

Ubicación	Versionada	Contenido y política
`scenarios/default/`	Sí	Insumos: TSV, red 3D, TAZ, plantilla `.sumocfg`, `nombres_vias.json`, programas de semáforo guardados.
`precalculadas/<preset>/`	Sí	`trips.geojson.gz` (11,7 MB comprimido frente a 63,7 MB), `volumenes.json`, `meta.json`. El `*.geojson` sin comprimir se ignora.
`data/processed/`	Sí	Evidencia descargada una vez: siniestros SDM, POI y edificios OSM/Microsoft.
`runs/<id>/`	No	Archivos generados por corrida y por comparación; se borran al cancelar (menos `estado.json`) o al cerrar la comparación.
`cache/`	No	`semaforos.json` y `priorizacion.json`, invalidados por mtime y tamaño de la red (y por `VERSION_ANALISIS`).
`runs/<id>/output/volumenes.json`	No	Caché de volúmenes junto a la corrida, invalidada por mtime/tamaño del `vehroute`.

Decisiones de almacenamiento relevantes:

- **Archivos pesados efímeros.** El FCD de una corrida completa es de gigabytes; se escribe cada 2 s (`device.fcd.period=2`), se convierte en streaming con `iterparse` y se borra tras el export. No se genera `emission-output` (2 GB sin uso).
- **Reproducibilidad de las precalculadas.** El `.gz` se escribe con `mtime=0`, de modo que regenerar el mismo contenido da el mismo archivo y Git no registra cambios espurios.
- **Servir comprimido.** La API entrega el `.gz` sin descomprimir con `Content-Encoding: gzip` y `X-Tamano-Original`, que el visor usa para dibujar el progreso de descarga.
- **Persistencia del historial.** `estado.json` se escribe como mucho cada 2 s durante una etapa; al reiniciar el servicio, `GestorCorridas` recarga las corridas existentes.
- **Las corridas no se pisan.** Los archivos generados van a `runs/<id>/`; sin `run_id` se escribe en el escenario (comportamiento heredado para ejecutar el pipeline desde la línea de comandos).

3.4 Interfaz de usuario

La interfaz es una aplicación web de una sola página a pantalla completa (`overflow: hidden`): el mapa de kepler.gl ocupa toda la ventana y los paneles flotan encima. Esta sección describe las pantallas de la interfaz gráfica y la navegación entre ellas; es consistente con los procesos de la Sección 2.3. Los colores por modo se definen en `modos.js` y se usan en el mapa, la leyenda y el panel.

3.4.1 Diagrama de navegación

El panel lateral tiene cuatro pestañas (el usuario puede pasar de cualquiera a cualquiera). Dos capas del mapa llevan a pestañas concretas, y el simulador 3D se abre desde «Semaforizar» y vuelve a ella al cerrarse [12].

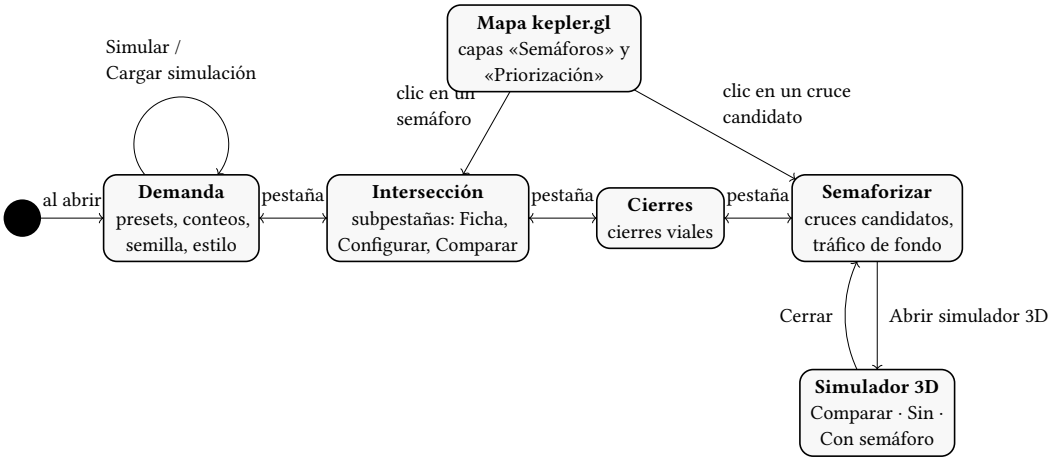


Figura 18: Diagrama de navegación de la interfaz gráfica.

Tabla 31: Transiciones del diagrama de navegación.

Transición	**Disparador**	**Proceso (Sección 2.3)**
Al abrir ⟶ Demanda	Se carga la aplicación; si existe la precalculada `encuesta`, se muestra automáticamente.	P1 (pasos 1–3, 8).
Demanda ⟶ Demanda	Botón «Simular» o «Cargar simulación» según coincida o no con una precalculada.	P1 (pasos 2–8).
Mapa ⟶ Intersección	Clic en un punto de la capa «Semáforos».	P2 (pasos 1–3).
Mapa ⟶ Semaforizar	Clic en un cruce de la capa «Priorización» o en la lista ordenada por puntaje.	P3 (pasos 1–2).
Semaforizar ↔ Simulador 3D	«Abrir simulador 3D» y «Cerrar».	P3 (pasos 3–12).
Entre pestañas	Clic en la pestaña.	—

3.4.2 Pantalla principal

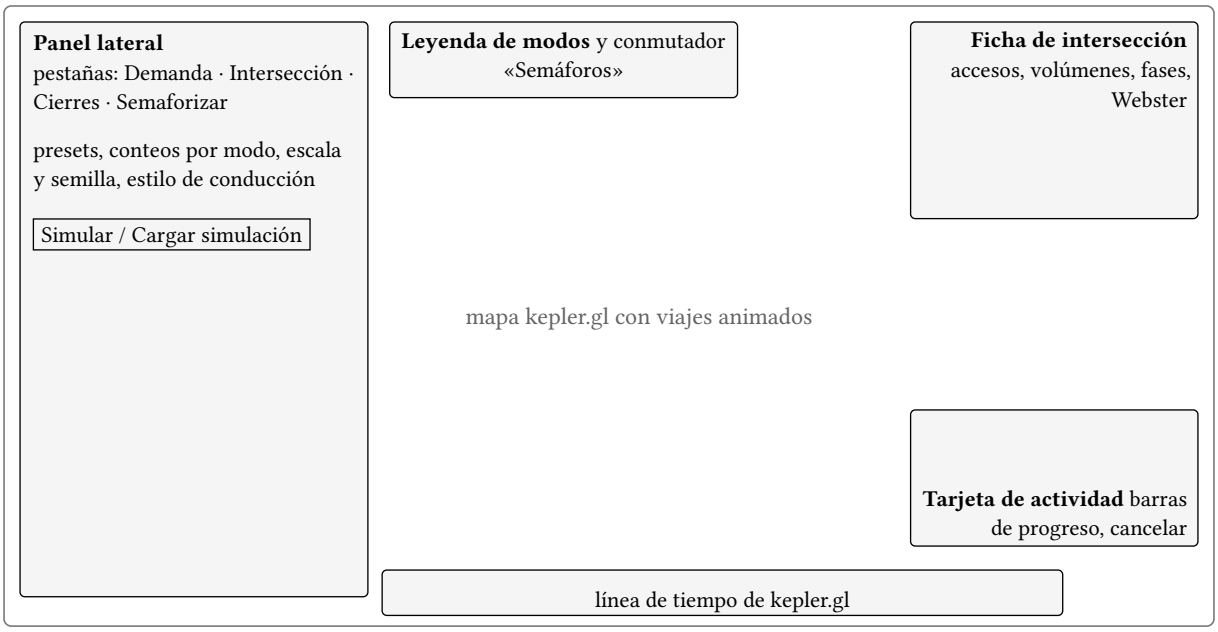


Figura 19: Esquema de la pantalla principal del visor.

Tabla 32: Elementos de la pantalla principal.

Elemento	Descripción
Panel de demanda	Elige un preset, ajusta viajes por modo (el total se valida contra el máximo de 100.000), escala, semilla y estilo. Si los valores coinciden con una precalculada, el botón principal es «Cargar simulación» y la carga es inmediata; si no, «Simular» lanza una corrida.
Panel de cierres	Lista de cierres viales (tramo y horario) que se envían con la solicitud de simulación.
Ficha de intersección	Pestaña «Intersección», con tres subpestañas: «Ficha» (accesos y volumen simulado por acceso, programa de fases), «Configurar» (editar duraciones de fase o calcular tiempos con Webster) y «Comparar» (lanza simulaciones con distintos programas y compara métricas de demora, cola y peatones).
Panel «Semaforizar»	Lista de cruces candidatos por puntaje, selección en el mapa, tráfico de fondo por modo y botón para abrir el simulador 3D.
Leyenda de modos	Colores por modo y conmutador de la capa «Semáforos».
Capas de intersecciones	«Semáforos» (puntos, con la ficha al hacer clic) y «Priorización» (cruces candidatos por nivel: prioritario, recomendado, observación). Las notas de la API (por ejemplo, «sin datos de siniestros») se muestran al usuario.
Tarjeta de actividad	Muestra la etapa actual, el avance global y el de la etapa, y permite cancelar. Al terminar una corrida bien, el visor carga sus trips solo. También muestra la barra de progreso de la descarga, calculada con `X-Tamano-Original` en las precalculadas.
Línea de tiempo	Control de kepler.gl para animar los viajes.

3.4.3 Simulador 3D

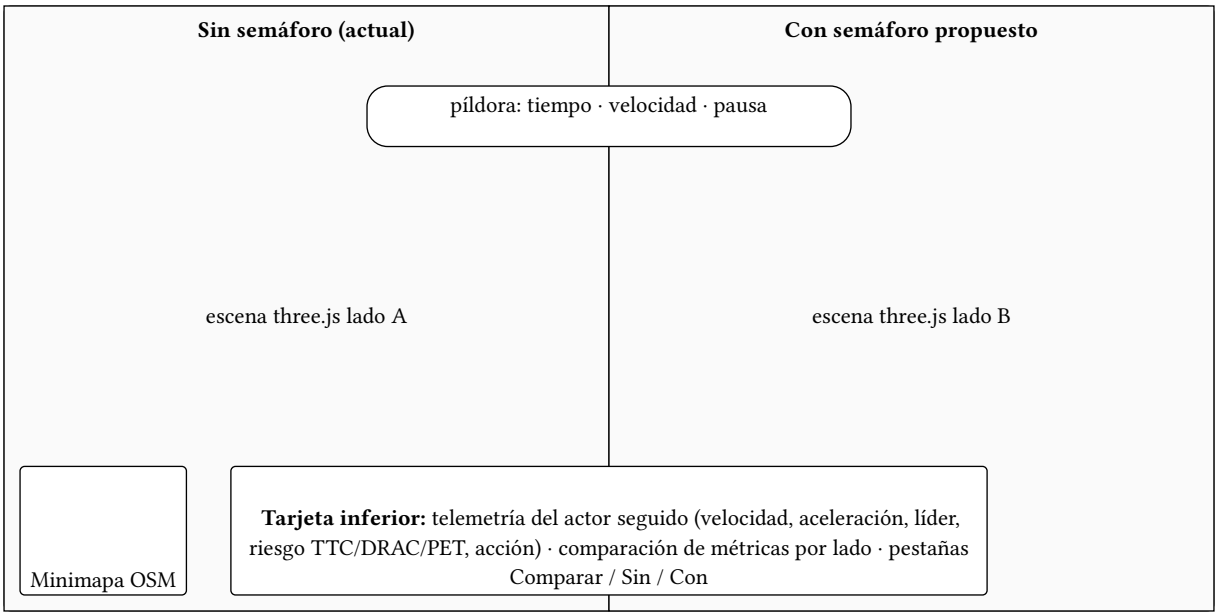


Figura 20: Esquema del simulador 3D comparativo.

Tabla 33: Elementos del simulador 3D.

Elemento	Descripción
Pantalla dividida	Pestañas «Comparar» (los dos lados), «Sin semáforo» y «Con semáforo» (un lado a pantalla completa). Cada lado es una escena three.js (`Escena3D`).
Píldora superior	Tiempo simulado, factor de velocidad (0,25–20) y pausa.
Tarjeta inferior	Telemetría del actor seguido y el índice de riesgo (TTC, DRAC, PET), con la acción en lenguaje natural, y las métricas comparadas por lado (vehículos en el cruce, espera, conflictos, riesgo máximo).
Actor seguido	Se puede cambiar o inyectar uno propio (auto, moto, taxi, bici, peatón, entre otros) eligiendo el acceso, el giro y el estilo de conducción.
Modo manual	El teclado acelera, frena y elige el giro; los comandos se aplican igual en los dos lados.
Cámara	De persecución, con colisión contra edificios, o aérea.
Minimapa	Mapa OSM con la posición y el rumbo del actor seguido (sin token).

El simulador se carga de forma diferida (`lazy`) porque three.js y maplibre pesan, y queda envuelto en `LimiteErrores`, de modo que un fallo gráfico no tumba el mapa.

4 Referencias

[1] IEEE, «IEEE Std 1016-2009 – IEEE Standard for Information Technology – Systems Design – Software Design Descriptions», 2009.

[2] ISO/IEC/IEEE, «ISO/IEC/IEEE 42010:2011 – Systems and software engineering – Architecture description», 2011.

[3] P. Kruchten, «Architectural Blueprints—The "4+1" View Model of Software Architecture», *IEEE Software*, vol. 12, n.º 6, pp. 42-50, 1995, [En línea]. Disponible en: http://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf

[4] IEEE, «IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications», 1998.

[5] IEEE, «IEEE Std 1058-1998 – IEEE Standard for Software Project Management Plans», 1998.

[6] Secretaría Distrital de Movilidad de Bogotá, «Encuesta de Movilidad de Bogotá 2023», Alcaldía Mayor de Bogotá, 2023.

[7] P. A. Lopez, M. Behrisch, L. Bieker-Walz, y J. Erdmann, «Microscopic Traffic Simulation using SUMO», en *21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2575-2582.

[8] F. V. Webster, «Traffic Signal Settings (Road Research Technical Paper No. 39)», Her Majesty's Stationery Office, 1958.

[9] D. Gettman, L. Pu, T. Sayed, y S. Shelby, «Surrogate Safety Assessment Model and Validation (FHWA-HRT-08-051)», Federal Highway Administration, 2008.

[10] Rational Software, «RUP. Software Architecture Document». [En línea]. Disponible en: http://www.ts.mah.se/RUP/RationalUnifiedProcess/webtmpl/templates/a_and_d/rup_sad.htm

[11] S. Ambler, «UML Data Modeling Profile». [En línea]. Disponible en: http://www.agiledata.org/essays/umlDataModelingProfile.html

[12] S. Ambler, «User Interface Flow Diagrams». [En línea]. Disponible en: http://www.agilemodeling.com/artifacts/uiFlowDiagram.htm